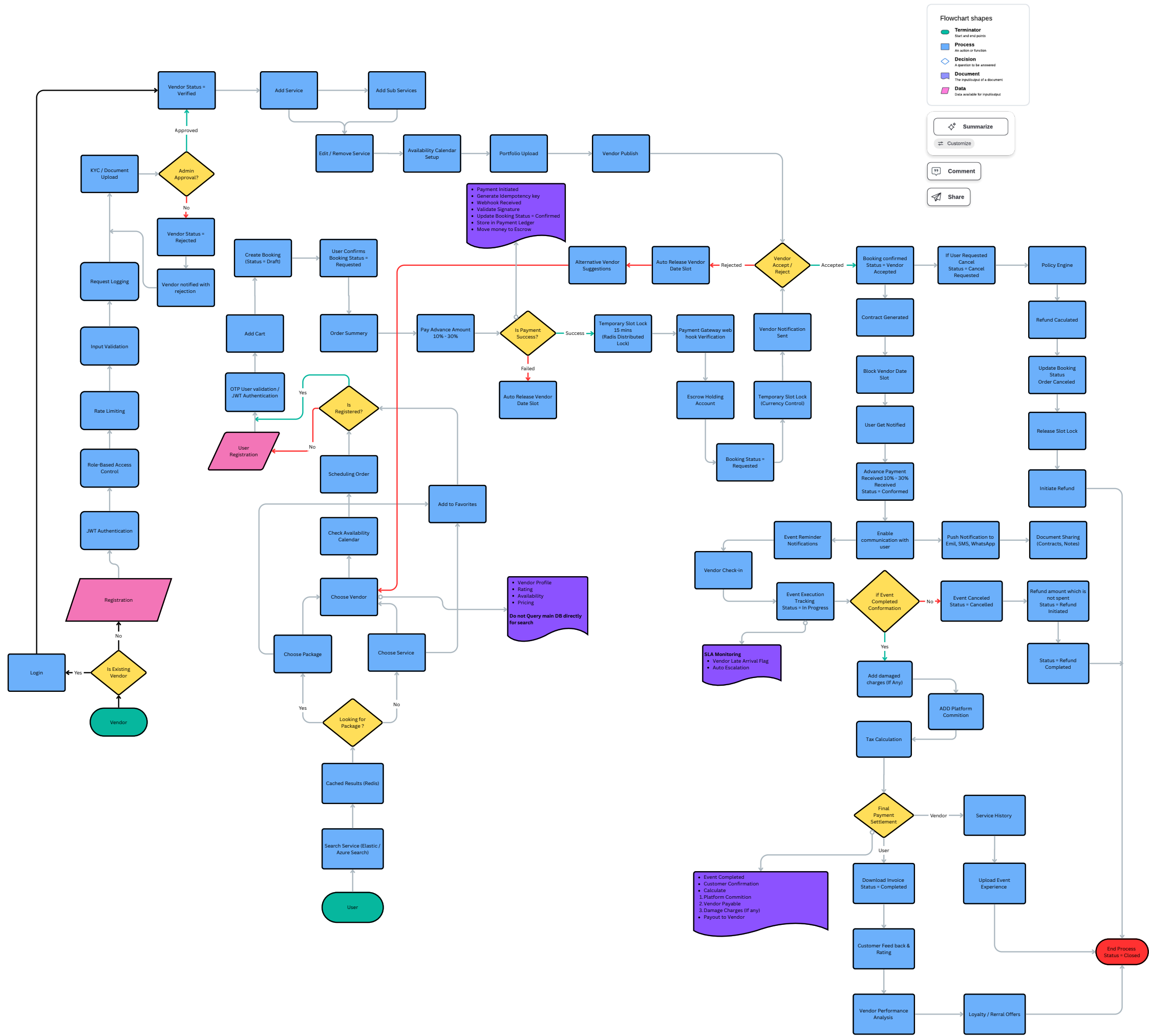




EVENT PLANNER PLATFORM

Full-Stack Development Blueprint

Angular 20	.NET Core 10	Azure Cloud	v1.0 Feb 2026
------------	--------------	-------------	---------------

Stack Layer	Technology	Version	Purpose
Frontend	Angular	20 (Signals + SSR)	User & Vendor Web Applications
Backend	.NET Core	10 (Minimal API)	REST APIs, CQRS, Domain Logic
Database	Azure SQL + Cosmos DB	Latest	Relational + Audit/Event Store
Cache	Azure Redis Enterprise	7.x	Slot Locking, Search Cache, Sessions
Search	Azure AI Search	Latest	Vendor & Service Discovery
Auth	Azure AD B2C	Latest	JWT, OTP, RBAC, Social SSO
Messaging	Azure Service Bus	Premium	Async Events, Dead-letter Queue
Realtime	Azure SignalR Service	Latest	Live Dashboard, Chat, Notifications
Serverless	Azure Durable Functions	v4	Timers, SLA, Workflows
Storage	Azure Blob + CDN	Latest	Portfolios, Contracts, Photos
DevOps	Azure DevOps	Latest	CI/CD, Boards, Repos, Test Plans

Confidential — Engineering Team Reference

1. Solution Architecture Overview

The Event Planner Platform is designed as a Modular Monolith on the backend and a standalone Angular 20 application on the frontend. The architecture prioritises developer velocity in early phases while enforcing clean domain boundaries that allow individual modules to be extracted into independent microservices when scaling pressure demands it.

1.1 Architecture Style



Backend: Modular Monolith with Vertical Slice Architecture

- **Single deployable .NET 10 application:** contains independently testable feature slices, each owning its own request, handler, validator, and repository
- **Domain modules are strictly isolated:** no direct cross-module references — all communication via MediatR in-process events or Azure Service Bus for async
- **Extraction-ready:** any module (e.g. Payment, Dispute) can be extracted to a standalone Azure Container App with a Service Bus interface change only
- **CQRS pattern enforced:** Commands mutate state; Queries are read-only and can bypass the write model to read replicas



Frontend: Angular 20 Standalone Architecture

- **Signal-based state management:** no NgRx store overhead for simple state; use Angular Signals for reactivity throughout
- **Feature-based folder structure:** each domain feature (bookings, vendors, events) is a standalone module with lazy-loaded routes
- **SSR with Angular Universal:** for SEO-sensitive pages (vendor profiles, service listings) using Angular 20 SSR
- **PWA for Vendor Mobile App:** vendors use a Progressive Web App for field check-in, deliverable upload, and earnings dashboard

1.2 System Topology

Layer	Component	Hosting	Scale Strategy
Edge	Azure Front Door + WAF	Azure Global	Geo-routing, DDoS, TLS termination
API Gateway	Azure API Management (APIM)	Azure Region	Rate limiting, JWT validation, versioning
Frontend	Angular 20 SPA + SSR	Azure Static Web Apps	CDN edge, auto-scale
Backend API	.NET 10 Minimal API	Azure Container Apps	Horizontal scale, min 2 replicas
Background Jobs	Azure Durable Functions	Consumption / Premium	Event-driven, timer-triggered
Primary DB	Azure SQL Hyperscale	Zone-redundant	Read replicas for queries
Audit/Events	Azure Cosmos DB	Multi-region	Change Feed for projections
Cache	Azure Redis Enterprise	Zone-redundant	Cluster mode, Redlock
Search	Azure AI Search (Standard)	Azure Region	Index replicas for throughput
Async Messaging	Azure Service Bus Premium	Zone-redundant	Partitioned topics, DLQ
File Storage	Azure Blob Storage (LRS)	Azure Region	CDN for media delivery
Realtime	Azure SignalR Service	Standard tier	Hub-based connections
Notifications	Azure Communication Services	Azure Global	Email, SMS, Push, WhatsApp

Monitoring	Azure Monitor + App Insights	Azure Region	Log Analytics workspace
------------	------------------------------	--------------	-------------------------

1.3 Module Map

The backend is organised into the following bounded-context modules. Each module owns its own tables, API endpoints, and domain logic:

Module	Responsibility	Key Entities	Publishes Events
Identity	Auth, registration, RBAC, OTP	User, Vendor, AdminUser, Role	UserRegistered, VendorVerified
Catalogue	Service categories, services, sub-services, pricing	ServiceCategory, Service, SubService, PriceRule	ServicePublished, PriceUpdated
Availability	Vendor calendars, slot locking, conflict detection	AvailabilitySlot, SlotLock	SlotLocked, SlotReleased
Events	Parent event entity, budget tracking, timeline	Event, EventTimeline, BudgetLine	EventCreated, EventCompleted
Bookings	Full booking lifecycle, state machine, contracts	VendorBooking, BookingAuditLog, Contract	BookingConfirmed, BookingCancelled
Payments	Payment initiation, webhooks, escrow, ledger	Payment, PaymentLedger, EscrowAccount, Outbox	PaymentReceived, EscrowReleased
Refunds	Policy engine, refund calculation, payout	RefundPolicy, RefundRequest	RefundInitiated, RefundCompleted
Disputes	Dispute lifecycle, evidence, admin arbitration	Dispute, DisputeEvidence, DisputeResolution	DisputeRaised, DisputeResolved
Settlement	Commission, tax, vendor payout, invoicing	Settlement, CommissionConfig, TaxRule, Invoice	VendorPaid, InvoiceGenerated
Reviews	Multi-dim ratings, moderation, vendor response	Review, ReviewDimension, ReviewResponse	ReviewPublished
Messaging	Thread-per-booking chat, notifications, prefs	MessageThread, Message, NotificationPref	MessageSent

Analytics	Platform KPIs, vendor performance, fraud flags	BookingMetric, VendorScore, FraudFlag	MetricRecorded
Admin	Admin portal aggregates all modules	AdminAction, AuditEntry	AdminActionTaken

2. Repository & Project Structure

Use a single Git monorepo managed with Azure DevOps. The monorepo keeps frontend and backend in one repository for atomic commits, shared type contracts, and unified CI/CD pipelines.

2.1 Monorepo Layout

```
event-planner/                                # Root monorepo

├─ .azure/                                    # Azure Bicep IaC templates

|   └─ main.bicep

|   └─ modules/                              # sql.bicep, redis.bicep, servicebus.bicep...

|   └─ environments/                         # dev.bicepparam, staging.bicepparam, prod.bicepparam

├─ src/

|   └─ backend/                              # .NET Core 10 Solution

|       └─ EventPlanner.sln

|       └─ src/

|           └─ EventPlanner.Api/             # ASP.NET Minimal API host

|           └─ EventPlanner.Domain/          # Entities, Value Objects, Domain Events

|           └─ EventPlanner.Application/     # CQRS Commands, Queries, Handlers

|           └─ EventPlanner.Infrastructure/  # EF Core, Redis, Azure SDK, Outbox

|           └─ EventPlanner.Modules.Identity/

|           └─ EventPlanner.Modules.Catalogue/
```

```
| | | └─ EventPlanner.Modules.Availability/

| | | └─ EventPlanner.Modules.Events/

| | | └─ EventPlanner.Modules.Bookings/

| | | └─ EventPlanner.Modules.Payments/

| | | └─ EventPlanner.Modules.Refunds/

| | | └─ EventPlanner.Modules.Disputes/

| | | └─ EventPlanner.Modules.Settlement/

| | | └─ EventPlanner.Modules.Reviews/

| | | └─ EventPlanner.Modules.Messaging/

| | | └─ EventPlanner.Modules.Analytics/

| | └─ tests/

| | └─ EventPlanner.UnitTests/

| | └─ EventPlanner.IntegrationTests/

| | └─ EventPlanner.ArchTests/ # ArchUnitNET boundary enforcement

└─ frontend/ # Angular 20 Workspace
```


2.2 Backend Module Internal Structure

Every module follows the same Vertical Slice folder convention. No exceptions. This keeps the codebase consistent and onboarding fast:

EventPlanner.Modules.Bookings/

└─ Features/

| └─ CreateBooking/

| | └─ CreateBookingCommand.cs # MediatR IRequest<Result<BookingId>>

| | └─ CreateBookingHandler.cs # IRequestHandler - all logic here

| | └─ CreateBookingValidator.cs # FluentValidation rules

| | └─ CreateBookingEndpoint.cs # Minimal API endpoint registration

| | └─ CreateBookingResponse.cs # Response DTO

| └─ GetBooking/

| └─ UpdateBookingStatus/

| └─ CancelBooking/

| └─ ListBookings/

└─ Domain/

| └─ Booking.cs # Aggregate root with state machine

| └─ BookingStatus.cs # Enum + transition guard methods

```
|   |— BookingAuditEntry.cs           # Domain event for audit log

|   |— Events/                       # Domain events: BookingConfirmed, etc.

|— Infrastructure/

|   |— BookingRepository.cs

|   |— BookingDbContext.cs           # Module-scoped DbContext

|   |— BookingConfiguration.cs       # EF Core Fluent API config

|— BookingsModule.cs                 # IHostedService registration
```

3. Database Design

Azure SQL Hyperscale is the primary OLTP store. Azure Cosmos DB (NoSQL) is used exclusively for the immutable audit log and event sourcing projections. Redis is used for caching and distributed locking — never as a primary store.

3.1 Core Table Schemas

users

```

CREATE TABLE users (

    user_id          UNIQUEIDENTIFIER  DEFAULT NEWSEQUENTIALID() PRIMARY KEY,

    full_name        NVARCHAR(150)     NOT NULL,

    email            NVARCHAR(255)     NOT NULL  UNIQUE,

    phone            NVARCHAR(20)       NOT NULL  UNIQUE,

    password_hash    NVARCHAR(500)     NULL,      -- NULL if SSO only

    city             NVARCHAR(100)     NULL,

    status           NVARCHAR(20)       NOT NULL  DEFAULT 'ACTIVE',  -- ACTIVE | SUSPENDED | DELETED

    created_at       DATETIMEOFFSET    NOT NULL  DEFAULT SYSDATETIMEOFFSET(),

    updated_at       DATETIMEOFFSET    NOT NULL  DEFAULT SYSDATETIMEOFFSET()

);

CREATE INDEX IX_users_email ON users(email);

CREATE INDEX IX_users_phone ON users(phone);

```

vendors

```
CREATE TABLE vendors (  
  
    vendor_id          UNIQUEIDENTIFIER  DEFAULT NEWSEQUENTIALID() PRIMARY KEY,  
  
    user_id            UNIQUEIDENTIFIER  NOT NULL REFERENCES users(user_id),  
  
    business_name       NVARCHAR(200)    NOT NULL,  
  
    gstin               NVARCHAR(20)     NULL,  
  
    kyc_status          NVARCHAR(20)     NOT NULL DEFAULT 'PENDING', --  
PENDING|UNDER_REVIEW|VERIFIED|REJECTED  
  
    insurance_url       NVARCHAR(500)    NULL,  
  
    insurance_expiry    DATE              NULL,  
  
    avg_rating          DECIMAL(3,2)     NOT NULL DEFAULT 0,  
  
    total_reviews       INT              NOT NULL DEFAULT 0,  
  
    is_published        BIT              NOT NULL DEFAULT 0,  
  
    created_at          DATETIMEOFFSET   NOT NULL DEFAULT SYSDATETIMEOFFSET(),  
  
    updated_at          DATETIMEOFFSET   NOT NULL DEFAULT SYSDATETIMEOFFSET()  
  
);
```

[service_categories](#) / [services](#) / [sub_services](#)

```

CREATE TABLE service_categories (

    category_id    UNIQUEIDENTIFIER    DEFAULT NEWSEQUENTIALID() PRIMARY KEY,


    name           NVARCHAR(100)       NOT NULL,           -- Photography, Catering, Music...


    slug           NVARCHAR(100)       NOT NULL    UNIQUE,


    icon_url       NVARCHAR(500)       NULL,


    is_active      BIT                  NOT NULL    DEFAULT 1


);


CREATE TABLE services (

    service_id     UNIQUEIDENTIFIER    DEFAULT NEWSEQUENTIALID() PRIMARY KEY,


    vendor_id      UNIQUEIDENTIFIER    NOT NULL    REFERENCES vendors (vendor_id),


    category_id    UNIQUEIDENTIFIER    NOT NULL    REFERENCES service_categories (category_id),


    name           NVARCHAR(200)       NOT NULL,


    description     NVARCHAR(2000)     NULL,


    base_price      DECIMAL(12,2)      NOT NULL,

```

```

pricing_type      NVARCHAR(20)      NOT NULL,    -- HOURLY | FLAT | PACKAGE

min_hours         INT                NULL,

is_active         BIT                NOT NULL    DEFAULT 1

);

CREATE TABLE sub_services (          -- NEW: Sub-service hierarchy

sub_service_id    UNIQUEIDENTIFIER  DEFAULT NEWSEQUENTIALID() PRIMARY KEY,

service_id        UNIQUEIDENTIFIER  NOT NULL    REFERENCES services(service_id),

name              NVARCHAR(200)     NOT NULL,    -- Drone Shoot, Live Stream, Album...

additional_price   DECIMAL(12,2)     NOT NULL    DEFAULT 0,

bundle_discount   DECIMAL(5,2)      NULL,        -- % discount when booked with parent

requires equip    BIT                NOT NULL    DEFAULT 0,

equipment_list     NVARCHAR(1000)    NULL,

lead_time_hours   INT                NOT NULL    DEFAULT 0,

is_active         BIT                NOT NULL    DEFAULT 1

```

);

events (Parent Entity — NEW)

```

CREATE TABLE events (
    -- NEW: Parent record for all vendor bookings

    event_id          UNIQUEIDENTIFIER  DEFAULT NEWSEQUENTIALID() PRIMARY KEY,

    user_id           UNIQUEIDENTIFIER  NOT NULL REFERENCES users(user_id),

    event_name        NVARCHAR(200)     NOT NULL,

    event_type        NVARCHAR(50)      NOT NULL,    -- WEDDING|BIRTHDAY|CORPORATE|RECEPTION

    event_date        DATE               NOT NULL,

    city              NVARCHAR(100)     NOT NULL,

    venue_details     NVARCHAR(500)     NULL,

    total_budget      DECIMAL(14,2)     NULL,

    committed_amount  DECIMAL(14,2)     NOT NULL DEFAULT 0,

    status            NVARCHAR(20)      NOT NULL DEFAULT 'DRAFT',

    created_at        DATETIMEOFFSET    NOT NULL DEFAULT SYSDATETIMEOFFSET()

);

```

vendor_bookings

```

CREATE TABLE vendor_bookings (

    booking_id          UNIQUEIDENTIFIER  DEFAULT NEWSEQUENTIALID() PRIMARY KEY,

    event_id            UNIQUEIDENTIFIER  NOT NULL  REFERENCES events(event_id),    -- Parent

    user_id             UNIQUEIDENTIFIER  NOT NULL  REFERENCES users(user_id),

    vendor_id           UNIQUEIDENTIFIER  NOT NULL  REFERENCES vendors(vendor_id),

    service_id          UNIQUEIDENTIFIER  NOT NULL  REFERENCES services(service_id),

    status              NVARCHAR(30)      NOT NULL  DEFAULT 'DRAFT',

    -- Status values: DRAFT|REQUESTED|VENDOR_ACCEPTED|CONFIRMED|IN_PROGRESS|

    --                  COMPLETED|SETTLED|DISPUTED|CANCEL_REQUESTED|CANCELLED|

    --                  REFUND_INITIATED|REFUND_COMPLETED|EXPIRED

    total_amount        DECIMAL(14,2)     NOT NULL,

    advance_amount       DECIMAL(14,2)     NOT NULL,

    advance_paid_at      DATETIMEOFFSET    NULL,

    contract_url         NVARCHAR(500)     NULL,

    idempotency_key      UNIQUEIDENTIFIER  NOT NULL  DEFAULT NEWID(),

```

```

created_at      DATETIMEOFFSET      NOT NULL      DEFAULT SYSDATETIMEOFFSET(),

updated_at      DATETIMEOFFSET      NOT NULL      DEFAULT SYSDATETIMEOFFSET()

);

CREATE TABLE booking_sub_services (    -- Many-to-many: booking ↔ sub_service

booking_id      UNIQUEIDENTIFIER NOT NULL REFERENCES vendor_bookings(booking_id),

sub_service_id  UNIQUEIDENTIFIER NOT NULL REFERENCES sub_services(sub_service_id),

price_at_booking  DECIMAL(12,2)      NOT NULL,

PRIMARY KEY (booking_id, sub_service_id)

);

```

payments & escrow

```

CREATE TABLE payment_ledger (

    ledger_id          UNIQUEIDENTIFIER  DEFAULT NEWSEQUENTIALID() PRIMARY KEY,

    booking_id         UNIQUEIDENTIFIER  NOT NULL REFERENCES vendor_bookings(booking_id),

    payment_type       NVARCHAR(20)      NOT NULL,    -- ADVANCE | FINAL | REFUND

    amount             DECIMAL(14,2)     NOT NULL,

    currency           CHAR(3)           NOT NULL     DEFAULT 'INR',

    gateway            NVARCHAR(50)      NOT NULL,    -- RAZORPAY | STRIPE | CASHFREE

    gateway_ref        NVARCHAR(200)     NULL,

    idempotency_key    NVARCHAR(200)     NOT NULL,

    status             NVARCHAR(20)      NOT NULL,    -- PENDING|SUCCESS|FAILED|REFUNDED

    created_at         DATETIMEOFFSET    NOT NULL     DEFAULT SYSDATETIMEOFFSET()

);

```

```

CREATE TABLE escrow_accounts (

    escrow_id          UNIQUEIDENTIFIER  DEFAULT NEWSEQUENTIALID() PRIMARY KEY,

```

```

booking_id          UNIQUEIDENTIFIER NOT NULL REFERENCES vendor_bookings(booking_id),

held_amount         DECIMAL(14,2)     NOT NULL,

platform_fee        DECIMAL(14,2)     NULL,

tax_amount          DECIMAL(14,2)     NULL,

damage_deduction    DECIMAL(14,2)     NOT NULL DEFAULT 0,

vendor_payable       DECIMAL(14,2)     NULL,

released_at         DATETIMEOFFSET    NULL,

released_to         NVARCHAR(10)      NULL          -- VENDOR | USER

);

```

```

CREATE TABLE outbox_messages (          -- Outbox Pattern for reliable messaging

```

```

message_id          UNIQUEIDENTIFIER DEFAULT NEWSEQUENTIALID() PRIMARY KEY,

correlation_id       NVARCHAR(200)     NOT NULL,

message_type         NVARCHAR(200)     NOT NULL,

payload             NVARCHAR(MAX)      NOT NULL,    -- JSON

```

```
created_at      DATETIMEOFFSET      NOT NULL  DEFAULT SYSDATETIMEOFFSET(),

processed_at    DATETIMEOFFSET      NULL,

retry_count     INT                NOT NULL  DEFAULT 0

);

CREATE INDEX IX_outbox_unprocessed ON outbox_messages(processed_at) WHERE processed_at IS NULL;
```

disputes & reviews

```

CREATE TABLE disputes (

    dispute_id          UNIQUEIDENTIFIER  DEFAULT NEWSEQUENTIALID() PRIMARY KEY,

    booking_id          UNIQUEIDENTIFIER  NOT NULL  REFERENCES vendor_bookings(booking_id),

    raised_by_user_id   UNIQUEIDENTIFIER  NOT NULL  REFERENCES users(user_id),

    dispute_type        NVARCHAR(30)      NOT NULL,    -- QUALITY|NO_SHOW|DAMAGE|PARTIAL

    description         NVARCHAR(2000)    NOT NULL,

    status              NVARCHAR(20)      NOT NULL  DEFAULT 'RAISED',

    -- RAISED|EVIDENCE_SUBMITTED|ADMIN_REVIEW|RESOLVED|ESCALATED

    resolution          NVARCHAR(20)      NULL,        -- FULL_REFUND|PARTIAL_REFUND|DISMISSED

    resolved_by         UNIQUEIDENTIFIER  NULL,        -- AdminUser FK

    resolved_at         DATETIMEOFFSET    NULL,

    created_at          DATETIMEOFFSET    NOT NULL  DEFAULT SYSDATETIMEOFFSET()

);

```

```

CREATE TABLE reviews (

```

```
review_id          UNIQUEIDENTIFIER  DEFAULT NEWSEQUENTIALID() PRIMARY KEY,

booking_id         UNIQUEIDENTIFIER  NOT NULL  REFERENCES vendor_bookings(booking_id),

user_id            UNIQUEIDENTIFIER  NOT NULL  REFERENCES users(user_id),

vendor_id          UNIQUEIDENTIFIER  NOT NULL  REFERENCES vendors(vendor_id),

overall_rating     TINYINT           NOT NULL  CHECK (overall_rating BETWEEN 1 AND 5),

punctuality        TINYINT           NULL      CHECK (punctuality BETWEEN 1 AND 5),

quality            TINYINT           NULL      CHECK (quality BETWEEN 1 AND 5),

communication      TINYINT           NULL      CHECK (communication BETWEEN 1 AND 5),

value_for_money    TINYINT           NULL      CHECK (value_for_money BETWEEN 1 AND 5),

comment            NVARCHAR(2000)     NULL,

vendor_response    NVARCHAR(1000)     NULL,

moderation_status  NVARCHAR(20)       NOT NULL  DEFAULT 'PENDING',

is_published       BIT                NOT NULL  DEFAULT 0,

created_at         DATETIMEOFFSET     NOT NULL  DEFAULT SYSDATETIMEOFFSET()
```

);

3.2 Cosmos DB — Audit Log Schema

Every booking status transition is written immutably to Cosmos DB. This is the compliance and dispute evidence store. Partition key: /bookingId.

}

4. API Design Standards

All APIs follow REST conventions with versioned routes, consistent error envelopes, cursor-based pagination, and idempotency keys on mutating operations. The OpenAPI spec in `/contracts/openapi.yaml` is the single source of truth — generated from .NET source attributes.

4.1 URL & Versioning Conventions

Base URL pattern

https://api.eventplanner.com/api/v1/{resource}

Resource patterns

GET	/api/v1/vendors	List vendors (paginated)
-----	-----------------	--------------------------

GET	/api/v1/vendors/{id}	Get vendor profile
-----	----------------------	--------------------

POST	/api/v1/vendors	Register vendor
------	-----------------	-----------------

PATCH	/api/v1/vendors/{id}/kyc	Update KYC status (Admin)
-------	--------------------------	---------------------------

GET	/api/v1/events	List user events
-----	----------------	------------------

POST	/api/v1/events	Create event (parent)
------	----------------	-----------------------

GET	/api/v1/events/{id}	Get event + all bookings
-----	---------------------	--------------------------

POST	/api/v1/bookings	Create booking (draft)
------	------------------	------------------------

PATCH	/api/v1/bookings/{id}/status	Transition state
-------	------------------------------	------------------

POST	/api/v1/bookings/{id}/cancel	Cancel with reason
POST	/api/v1/payments/initiate	Initiate payment
POST	/api/v1/payments/webhook	Gateway webhook (unsigned)
POST	/api/v1/disputes	Raise dispute
PATCH	/api/v1/disputes/{id}/resolve	Admin resolution
POST	/api/v1/reviews	Submit review
PATCH	/api/v1/reviews/{id}/respond	Vendor response
GET	/api/v1/search/vendors	Search vendors
GET	/api/v1/search/services	Search services + sub-services

4.2 Standard Response Envelope

```
// SUCCESS RESPONSE
```

```
{  
  
  "success": true,  
  
  "data": { ... },           // Actual payload  
  
  "meta": {  
  
    "requestId": "trace-id",    // X-Request-ID header echoed  
  
    "timestamp": "2026-02-20T14:30:00Z"  
  
  }  
  
}
```

```
// PAGINATED RESPONSE
```

```
{  
  
  "success": true,  
  
  "data": [ ... ],  
  
  "pagination": {
```

```
"cursor": "base64-encoded-cursor", // Cursor-based, NOT offset

"hasMore": true,

"pageSize": 20

}

}

// ERROR RESPONSE

{

"success": false,

"error": {

"code": "BOOKING_SLOT_UNAVAILABLE",

"message": "The selected time slot is no longer available.",

"details": [ { "field": "slotId", "issue": "Slot expired" } ],

"requestId": "trace-id"

}
```

```
}
```

4.3 Idempotency

All POST endpoints that create bookings or initiate payments MUST require an Idempotency-Key header. The server stores the response for 24 hours and returns the same response for duplicate requests with the same key.

```
POST /api/v1/payments/initiate
```

```
Headers:
```

```
Authorization: Bearer {jwt}
```

```
Idempotency-Key: {uuid-v4}           // Client-generated, unique per attempt
```

```
Content-Type: application/json
```

```
// .NET Handler checks outbox/idempotency table before processing
```

```
// If key exists: return cached response (202 Accepted)
```

```
// If key new: process and store response in idempotency table
```


5. Backend Implementation Guide (.NET Core 10)

5.1 Project Setup & Key Packages

```
# Create solution
```

```
dotnet new sln -n EventPlanner
```

```
dotnet new webapi -n EventPlanner.Api --use-minimal-apis
```

```
dotnet new classlib -n EventPlanner.Domain
```

```
dotnet new classlib -n EventPlanner.Application
```

```
dotnet new classlib -n EventPlanner.Infrastructure
```

```
# Essential NuGet packages
```

```
dotnet add package MediatR # CQRS dispatcher
```

```
dotnet add package FluentValidation.AspNetCore # Request validation
```

```
dotnet add package Microsoft.EntityFrameworkCore # ORM
```

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
```

```
dotnet add package MassTransit.AzureServiceBus # Messaging
```

```
dotnet add package StackExchange.Redis # Redis client
```

```
dotnet add package Azure.Storage.Blobs # Blob storage
```

```
dotnet add package Azure.Search.Documents           # AI Search client

dotnet add package Azure.Security.KeyVault.Secrets

dotnet add package Azure.Identity                   # Managed Identity

dotnet add package Polly.Extensions.Http             # Resilience policies

dotnet add package Serilog.AspNetCore                # Structured logging

dotnet add package ArchUnitNET                       # Architecture tests

dotnet add package Bogus                            # Test data generation

dotnet add package Microsoft.AspNetCore.SignalR     # Real-time hub
```

5.2 Vertical Slice — Complete Example (Create Booking)

CreateBookingCommand.cs

```
using MediatR;

using EventPlanner.Domain.Bookings;

namespace EventPlanner.Modules.Bookings.Features.CreateBooking;

public record CreateBookingCommand(

    Guid EventId,

    Guid VendorId,

    Guid ServiceId,

    IReadOnlyList<Guid> SubServiceIds,

    DateOnly EventDate,

    TimeOnly StartTime,

    TimeOnly EndTime

) : IRequest<Result<BookingId>>;
```



```
using FluentValidation;

public class CreateBookingValidator : AbstractValidator<CreateBookingCommand>
{
    public CreateBookingValidator()
    {
        RuleFor(x => x.EventId).NotEmpty();

        RuleFor(x => x.VendorId).NotEmpty();

        RuleFor(x => x.ServiceId).NotEmpty();

        RuleFor(x => x.EventDate).GreaterThan(DateOnly.FromDateTime(DateTime.UtcNow))

            .WithMessage("Event date must be in the future.");

        RuleFor(x => x.StartTime).LessThan(x => x.EndTime)

            .WithMessage("Start time must be before end time.");
    }
}
```



CreateBookingHandler.cs

```
public class CreateBookingHandler : IRequestHandler<CreateBookingCommand, Result<BookingId>>

{

    private readonly IBookingRepository _bookings;

    private readonly IAvailabilityService _availability; // Cross-module via interface

    private readonly IPricingService _pricing;

    private readonly IOutboxService _outbox;

    private readonly IAuditLogService _audit;

    public async Task<Result<BookingId>> Handle(

        CreateBookingCommand cmd, CancellationToken ct)

    {

        // 1. Verify slot is available (calls Availability module)

        var lockResult = await _availability.TryAcquireSlotLockAsync(

            cmd.VendorId, cmd.EventDate, cmd.StartTime, cmd.EndTime, ct);

        if (!lockResult.IsSuccess) return Result.Fail("Slot no longer available");
```

```
// 2. Calculate pricing

var price = await _pricing.CalculateTotalAsync(

    cmd.ServiceId, cmd.SubServiceIds, ct);

// 3. Create aggregate root

var booking = Booking.Create(

    cmd.EventId, cmd.VendorId, cmd.ServiceId,

    cmd.SubServiceIds, price, cmd.EventDate);

// 4. Persist + publish outbox event atomically

await _bookings.AddAsync(booking, ct);

await _outbox.PublishAsync(new BookingCreatedEvent(booking.Id), ct);

// 5. Audit log (Cosmos DB)
```

```
        await _audit.RecordAsync(new AuditEntry(

            booking.Id, "CREATED", null, "DRAFT", Actor.FromContext()), ct);

    }

    return Result.Ok(booking.Id);

}

}
```

CreateBookingEndpoint.cs (Minimal API)

```
public static class CreateBookingEndpoint

{

    public static void MapCreateBooking(this IEndpointRouteBuilder app)

    {

        app.MapPost("/api/v1/bookings", async (

            CreateBookingRequest req,

            IMediator mediator,

            HttpContext ctx,

            CancellationToken ct) =>

        {

            var command = new CreateBookingCommand(

                req.EventId, req.VendorId, req.ServiceId,

                req.SubServiceIds, req.EventDate, req.StartTime, req.EndTime);

            var result = await mediator.Send(command, ct);
```

```

        return result.IsSuccess

        ? Results.Created($"{api/v1/bookings/{result.Value}", new { bookingId =
result.Value })

        : Results.BadRequest(ApiError.From(result));

    })

    .RequireAuthorization("UserRole")

    .AddEndpointFilter<IdempotencyFilter>()

    .AddEndpointFilter<ValidationFilter<CreateBookingRequest>>()

    .WithName("CreateBooking")

    .WithOpenApi();

}

}

```

5.3 Booking State Machine (Domain Aggregate)

```
public class Booking : AggregateRoot

{

    public BookingStatus Status { get; private set; }

    public Result TransitionTo(BookingStatus newStatus, Actor actor)

    {

        if (!AllowedTransitions[Status].Contains(newStatus))

            return Result.Fail($"Cannot move from {Status} to {newStatus}");

        var oldStatus = Status;

        Status = newStatus;

        UpdatedAt = DateTimeOffset.UtcNow;

        AddDomainEvent(new BookingStatusChangedEvent(

            Id, oldStatus, newStatus, actor, DateTimeOffset.UtcNow));
    }
}
```

```
return Result.Ok();

}

private static readonly Dictionary<BookingStatus, BookingStatus[]> AllowedTransitions = new()

{

    [BookingStatus.Draft] = [BookingStatus.Requested],

    [BookingStatus.Requested] = [BookingStatus.VendorAccepted, BookingStatus.Expired],

    [BookingStatus.VendorAccepted] = [BookingStatus.Confirmed],

    [BookingStatus.Confirmed] = [BookingStatus.InProgress,
BookingStatus.CancelRequested],

    [BookingStatus.InProgress] = [BookingStatus.Completed],

    [BookingStatus.Completed] = [BookingStatus.Settled, BookingStatus.Disputed],

    [BookingStatus.Disputed] = [BookingStatus.Settled],

    [BookingStatus.CancelRequested] = [BookingStatus.Cancelled],

    [BookingStatus.Cancelled] = [BookingStatus.RefundInitiated],
```

```
        [BookingStatus.RefundInitiated] = [BookingStatus.RefundCompleted],  
  
        };  
  
    }
```

5.4 Outbox Worker (Reliable Messaging)

```
public class OutboxWorker : BackgroundService

{

    protected override async Task ExecuteAsync(CancellationToken ct)

    {

        while (!ct.IsCancellationRequested)

        {

            // Poll unprocessed outbox messages every 5 seconds

            var messages = await _db.OutboxMessages

                .Where(m => m.ProcessedAt == null && m.RetryCount < 5)

                .OrderBy(m => m.CreatedAt)

                .Take(50)

                .ToListAsync(ct);

            foreach (var msg in messages)

            {
```

```
        try {

            await _serviceBus.PublishAsync(msg.MessageType, msg.Payload, ct);

            msg.ProcessedAt = DateTimeOffset.UtcNow;

        } catch {

            msg.RetryCount++;

        }

    }

    await _db.SaveChangesAsync(ct);

    await Task.Delay(TimeSpan.FromSeconds(5), ct);

}

}

}
```

6. Frontend Implementation Guide (Angular 20)

6.1 Angular Project Setup

```
# Install Angular CLI 20
```

```
npm install -g @angular/cli@20
```

```
# Create workspace with three apps
```

```
ng new event-planner --no-create-application
```

```
cd event-planner
```

```
ng generate application user-portal --routing --style=scss --standalone
```

```
ng generate application vendor-portal --routing --style=scss --standalone
```

```
ng generate application admin-portal --routing --style=scss --standalone
```

```
# Generate shared libraries
```

```
ng generate library ui # Design system
```

```
ng generate library data-access # HTTP services
```

```
ng generate library domain # Shared interfaces & enums
```

```
ng generate library utils                                # Pipes, guards, interceptors

# Key dependencies

npm install @azure/msal-angular @azure/msal-browser    # Azure AD B2C auth

npm install @microsoft/signalr                          # Real-time

npm install @angular/pwa                               # PWA for vendor app

npm install date-fns                                    # Date utilities

npm install echarts ngx-echarts                        # Charts/analytics
```

6.2 Feature Folder Structure (User Portal)

```
apps/user-portal/src/
```

```
└─ app/
```

```
|   └─ app.config.ts           # Standalone app config (no AppModule)
```

```
|   └─ app.routes.ts          # Root lazy routes
```

```
|   └─ core/
```

```
|   |   └─ interceptors/
```

```
|   |   |   └─ auth.interceptor.ts    # Attach JWT to every request
```

```
|   |   |   └─ idempotency.interceptor.ts
```

```
|   |   |   └─ error.interceptor.ts
```

```
|   |   └─ guards/
```

```
|   |   |   └─ auth.guard.ts
```

```
|   |   |   └─ role.guard.ts
```

```
|   |   └─ services/
```

```
|   |       └─ notification.service.ts # SignalR connection
```

```
|   └─ features/
```

```
|   └─ auth/          (login, register, otp)

|   └─ events/        (create-event, event-dashboard, budget-tracker)

|   └─ discovery/     (search, vendor-profile, sub-service-selector)

|   └─ bookings/      (order-summary, booking-list, booking-detail)

|   └─ payments/      (payment-gateway, payment-status)

|   └─ disputes/      (raise-dispute, evidence-upload)

|   └─ reviews/       (write-review, view-reviews)

|   └─ messages/      (chat-thread, notifications)
```

6.3 Signal-Based State — Booking Store

```
// libs/data-access/src/lib/booking.store.ts

import { signalStore, withState, withMethods, patchState } from "@ngrx/signals";

import { inject } from "@angular/core";

import { BookingService } from "../booking.service";

type BookingState = {

  bookings: VendorBooking[];

  selected: VendorBooking | null;

  loading: boolean;

  error: string | null;

};

const initialState: BookingState = {

  bookings: [], selected: null, loading: false, error: null

};
```

```
export const BookingStore = signalStore(

  { providedIn: "root" },

  withState(initialState),

  withMethods((store, svc = inject(BookingService)) => ({

    async loadBookings(eventId: string) {

      patchState(store, { loading: true });

      const result = await svc.getByEvent(eventId);

      patchState(store, { bookings: result.data, loading: false });

    },

    async createBooking(cmd: CreateBookingRequest) {

      patchState(store, { loading: true });

      const result = await svc.create(cmd);

      patchState(store, s => ({

        bookings: [...s.bookings, result.data],
```

```
        loading: false

    });

    return result;

}

}))

);
```

6.4 HTTP Service with Idempotency

```
// libs/data-access/src/lib/booking.service.ts

@Injectable({ providedIn: "root" })

export class BookingService {

  private http = inject(HttpClient);

  private base = "/api/v1/bookings";

  create(cmd: CreateBookingRequest): Observable<ApiResponse<{ bookingId: string }>> {

    const idempotencyKey = crypto.randomUUID();

    return this.http.post<ApiResponse<{ bookingId: string }>>(this.base, cmd, {

      headers: { "Idempotency-Key": idempotencyKey }

    });

  }

  updateStatus(id: string, status: BookingStatus): Observable<ApiResponse<void>> {

    return this.http.patch<ApiResponse<void>>(`${this.base}/${id}/status`, { status });

  }

}
```

```
}  
  
}
```

6.5 Real-Time Event Dashboard (SignalR)

```
// core/services/event-hub.service.ts

@Injectable({ providedIn: "root" })

export class EventHubService {

    private connection: HubConnection;

    private bookingUpdates$ = new Subject<BookingStatusUpdate>();

    connect(eventId: string): void {

        this.connection = new HubConnectionBuilder()

            .withUrl(`/hubs/events?eventId=${eventId}`, {

                accessTokenFactory: () => this.auth.getToken()

            })

            .withAutomaticReconnect()

            .build();

        this.connection.on("BookingStatusUpdated", (update: BookingStatusUpdate) => {
```

```
        this.bookingUpdates$.next(update);

    });

    this.connection.on("VendorCheckedIn", (vendorId: string) => {

        // Update event day dashboard in real-time

    });

    this.connection.start();

}

onBookingUpdate(): Observable<BookingStatusUpdate> {

    return this.bookingUpdates$.asObservable();

}

}
```

6.6 Route Configuration (Lazy Loading)

```
// app.routes.ts

export const routes: Routes = [

  { path: "", loadComponent: () => import("../features/home/home.component") },

  {

    path: "events",

    canActivate: [authGuard],

    children: [

      { path: "", loadComponent: () => import("../features/events/event-list.component") },

      { path: "new", loadComponent: () => import("../features/events/create-event.component") },

      { path: ":id", loadComponent: () => import("../features/events/event-dashboard.component") },

    ]

  },

  {

    path: "vendors/:id",

    loadComponent: () => import("../features/discovery/vendor-profile.component")
```

```
    },  
  
    {  
  
      path: "bookings",  
  
      canActivate: [authGuard],  
  
      loadChildren: () => import("./features/bookings/bookings.routes")  
  
    },  
  
    {  
  
      path: "admin",  
  
      canActivate: [authGuard, roleGuard("Admin")],  
  
      loadChildren: () => import("./features/admin/admin.routes")  
  
    },  
  
  ],  
  
];
```

7. Azure Infrastructure as Code (Bicep)

All Azure resources are provisioned using Azure Bicep templates stored in the `.azure/` folder. Never create resources manually in the portal — everything is code-controlled and environment-parameterised.

7.1 Resource Naming Convention

Resource Type	Pattern	Example
Resource Group	rg-{app}-{env}	rg-eventplanner-prod
App Service / Container App	ca-{app}-api-{env}	ca-eventplanner-api-prod
Azure SQL Server	sql-{app}-{env}	sql-eventplanner-prod
Azure Redis	redis-{app}-{env}	redis-eventplanner-prod
Service Bus Namespace	sb-{app}-{env}	sb-eventplanner-prod
Storage Account	st{app}{env}	steventplannerprod
AI Search	srch-{app}-{env}	srch-eventplanner-prod
Cosmos DB Account	cosmos-{app}-{env}	cosmos-eventplanner-prod
Key Vault	kv-{app}-{env}	kv-eventplanner-prod
App Insights	appi-{app}-{env}	appi-eventplanner-prod
Container Registry	cr{app}{env}	creventplannerprod
SignalR Service	sigr-{app}-{env}	sigr-eventplanner-prod

7.2 Core Bicep Template (main.bicep)

```
param environment string                // dev | staging | prod

param location string = resourceGroup().location

param sqlAdminPassword string          // from Key Vault reference


// Container App for .NET API

module api './modules/container-app.bicep' = {

    name: 'api-deployment'

    params: {

        name: 'ca-eventplanner-api-${environment}'

        image: 'creventplanner.azurecr.io/api:${deploymentTag}'

        env: environment

        minReplicas: environment == 'prod' ? 2 : 1

        maxReplicas: 20

        secrets: [{ name: 'sqlconn', keyVaultUrl: kv.outputs.sqlConnSecret }]

    }

}
```

```
}

// Azure SQL with read replica in prod

module sql './modules/sql.bicep' = {

    name: 'sql-deployment'

    params: {

        serverName: 'sql-eventplanner-${environment}'

        databaseName: 'EventPlannerDb'

        sku: environment == 'prod' ? 'Hyperscale' : 'GeneralPurpose'

        enableReadReplica: environment == 'prod'

    }

}

// Service Bus with dead-letter topic per domain event

module sb './modules/service-bus.bicep' = {
```

```
name: 'sb-deployment'

params: {

  namespaceName: 'sb-eventplanner-${environment}'

  sku: environment == 'prod' ? 'Premium' : 'Standard'

  topics: ['bookings', 'payments', 'notifications', 'disputes']

}

}
```

7.3 Service Bus Topics & Subscriptions

Topic	Publisher	Subscriptions (Consumers)	DLQ Retention
bookings	Bookings Module	Notifications, Availability, Analytics	7 days
payments	Payments Module	Bookings (confirm), Escrow, Notifications	14 days
notifications	All modules	Notification Hub Worker	3 days
disputes	Disputes Module	Admin, Notifications, Settlement	14 days
vendor-events	Identity Module	Catalogue Indexer, Notifications	7 days
outbox-relay	Outbox Worker	Domain-specific topic routers	3 days

8. CI/CD Pipeline (Azure DevOps)

8.1 Pipeline Strategy

Branch Strategy (GitHub Flow)

- **main**: Production branch — protected, requires 2 approvals + all checks passing
- **develop**: Integration branch — auto-deploys to dev environment on merge
- **feature/EP-{ticket}-{slug}**: Feature branches — PR to develop, auto-cancelled after 14 days
- **hotfix/EP-{ticket}-{slug}**: Hotfix branches — PR directly to main + backport to develop
- **release/v{major}.{minor}**: Release branches — used for staging validation before main merge

8.2 Pipeline Stages

Stage	Trigger	Steps	Gate
CI — Build & Test	Every PR + push to develop/main	Restore, Build, Unit Tests, Integration Tests, ArchUnit Tests, SAST (SonarCloud), Docker Build	All tests green, coverage ≥ 80%, no critical SAST issues
CD — Dev Deploy	Merge to develop	Push image to ACR, Deploy to dev Container App, Run smoke tests, Seed dev data	Smoke test pass
CD — Staging Deploy	Merge to release/*	Deploy to staging, Run E2E tests (Playwright), Performance test (k6), OWASP ZAP scan	E2E green, p95 latency < 800ms, no high OWASP findings
CD — Prod Deploy	Merge to main + manual approval	Blue-green deploy to Container Apps, Health check, Swap traffic 10% → 50% → 100%, Smoke test	Manual gate: 2 approvers, SRE sign-off, post-deploy health check
DB Migrations	Part of CD stages	EF Core migrations run before app swap via migration container job	Migration idempotency verified, rollback script prepared

8.3 Pipeline YAML (Backend CI)

```
# .azuredevops/backend-ci.yml

trigger:

    branches: { include: [develop, main, feature/*, hotfix/*] }

variables:

    dotnetVersion: "10.x"

    buildConfig: Release

stages:

- stage: Build

    jobs:

    - job: BuildAndTest

        pool: { vmImage: ubuntu-latest }

        steps:

        - task: UseDotNet@2
```

```
inputs: { version: $(dotnetVersion) }
```

```
- script: dotnet restore
```

```
displayName: Restore packages
```

```
- script: dotnet build --no-restore -c $(buildConfig)
```

```
displayName: Build
```

```
- script: dotnet test --no-build -c $(buildConfig)
```

```
--collect "XPlat Code Coverage" --logger trx
```

```
displayName: Unit + Integration Tests
```

```
- task: PublishTestResults@2
```

```
inputs: { testResultsFormat: VSTest, testResultsFiles: "**/*.trx" }
```

```
- task: PublishCodeCoverageResults@1
```

```
inputs: { codeCoverageTool: Cobertura, summaryFileLocation: "coverage.xml" }
```

```
- task: Docker@2
```

```
displayName: Build & Push Docker Image
```

```
inputs:
```

```
containerRegistry: $(acrServiceConnection)
```

```
repository: eventplanner/api
```

```
command: buildAndPush
```

```
tags: $(Build.BuildId)
```

9. Security Implementation Checklist

9.1 Authentication & Authorisation

Item	Implementation	Priority
JWT Access Tokens	Issued by Azure AD B2C, 15-minute lifetime, RS256 signed	P0
Refresh Tokens	Rotating refresh tokens, 7-day lifetime, single-use	P0
Token Revocation	Redis blocklist for revoked JTIs on logout/suspicious activity	P0
RBAC	Claims-based: Role = User Vendor Admin SuperAdmin	P0
Resource-level Auth	Users can only access their own events/bookings (owner check in handlers)	P0
OTP Verification	Phone verified before any booking. 6-digit, 5-minute TTL, max 3 attempts	P0
Social SSO	Google + Apple via Azure AD B2C external identity providers	P1
MFA for Admin	Admin portal requires MFA via Azure AD B2C conditional access	P0

9.2 API Security

Item	Implementation	Priority
Rate Limiting	Azure APIM: 100 req/min per IP, 1000/min per authenticated user	P0
Input Validation	FluentValidation on ALL commands; APIM schema validation on entry	P0
SQL Injection	EF Core parameterised queries only. Raw SQL prohibited in code review	P0
Webhook Signature	HMAC-SHA256 validation on all payment gateway webhooks	P0
CORS	Strict origin whitelist — only portal domains allowed	P0
HTTPS Only	Azure Front Door enforces HTTPS. HTTP → 301 redirect	P0
WAF Rules	Azure Front Door WAF: OWASP 3.2 rule set + custom rules for payment endpoints	P0
Secrets Management	All connection strings in Azure Key Vault. Never in appsettings or env vars in code	P0
Blob SAS Tokens	Short-lived SAS tokens (1 hour) for user file uploads. No public blob containers	P0
DDoS Protection	Azure DDoS Standard on the virtual network. Front Door absorbs volumetric attacks	P1

9.3 Data Protection

PII & Financial Data Handling

- **Encrypt at rest:** Azure SQL TDE (enabled by default), Cosmos DB encryption, Key Vault-managed keys
- **Encrypt in transit:** TLS 1.2 minimum across all service-to-service connections
- **PII masking in logs:** Serilog destructuring policies mask email, phone, and payment references in all log output
- **Payment card data:** Never stored locally. All card data handled by PCI-compliant payment gateway
- **GDPR compliance:** User data deletion endpoint within 30 days. Data export available. Consent recorded

10. Testing Strategy

Test Type	Tool	Target Coverage	Runs In
Unit Tests	xUnit + FluentAssertions + Bogus	Domain logic, handlers, validators ≥ 90%	Every PR (< 2 min)
Integration Tests	xUnit + Testcontainers-dotnet	API endpoints with real SQL/Redis ≥ 70%	Every PR (< 8 min)
Architecture Tests	ArchUnitNET	Module boundary enforcement, no circular deps	Every PR (< 1 min)
Contract Tests	Pact.NET	Frontend ↔ Backend API contracts	Every PR
E2E Tests	Playwright + TypeScript	Critical user journeys: Register → Book → Pay	Staging deploy (< 15 min)
Performance Tests	k6	500 concurrent users, p95 < 800ms, booking flow	Staging deploy
Security Tests	OWASP ZAP + SonarCloud SAST	API endpoints, dependency audit	Staging deploy
Accessibility	axe-core + Playwright	WCAG 2.1 AA on all Angular views	Staging deploy

10.1 Integration Test Pattern (Testcontainers)

```
public class BookingIntegrationTests : IAsyncLifetime

{

    private readonly MsSqlContainer _sqlContainer = new MsSqlBuilder()

        .WithImage("mcr.microsoft.com/mssql/server:2022-latest")

        .Build();

    private readonly RedisContainer _redisContainer = new RedisBuilder().Build();

    private WebApplicationFactory<Program> _factory;

    public async Task InitializeAsync()

    {

        await _sqlContainer.StartAsync();

        await _redisContainer.StartAsync();

        _factory = new WebApplicationFactory<Program>().WithWebHostBuilder(b =>
```

```
{

    b.ConfigureServices(s => {

        s.AddDbContext<AppDbContext>(o =>

            o.UseSqlServer(_sqlContainer.GetConnectionString()));

        s.AddStackExchangeRedisCache(o =>

            o.Configuration = _redisContainer.GetConnectionString());

    });

});

}
```

[Fact]

```
public async Task CreateBooking_ValidRequest_ReturnsCreated()
```

```
{

    var client = _factory.CreateClient();

    client.DefaultRequestHeaders.Authorization =
```

```
new AuthenticationHeaderValue("Bearer", await GetTestJwt("user"));

var response = await client.PostAsJsonAsync("/api/v1/bookings", new {

    EventId = Guid.NewGuid(), VendorId = Guid.NewGuid(),

    ServiceId = Guid.NewGuid(), SubServiceIds = Array.Empty<Guid>(),

    EventDate = DateOnly.FromDateTime(DateTime.UtcNow.AddDays(30))

}, new HttpRequestMessage().Headers.Add("Idempotency-Key", Guid.NewGuid().ToString()));

response.StatusCode.Should().Be(HttpStatusCode.Created);

}

}
```

11. Development Sprint Plan

Based on a 2-week sprint cadence with a team of 3 backend developers, 2 frontend developers, 1 DevOps engineer, and 1 QA engineer. Total estimated timeline to production-ready MVP: 24 weeks (12 sprints).

11.1 Sprint Breakdown

Sprint	Weeks	Theme	Key Deliverables	Done When
S1–S2	1–4	Foundation	Azure infra (Bicep), CI/CD pipeline, monorepo setup, DB schema v1, auth module, APIM config, dev environment runbook	Engineers can run full stack locally and deploy to dev
S3–S4	5–8	Vendor Platform	KYC flow, background check integration, service/sub-service catalogue, availability calendar, portfolio upload, vendor portal Angular shell	A vendor can register, get approved, and publish services
S5–S6	9–12	User Booking Flow	Event creation, multi-vendor search + filter, sub-service selector, slot locking (Redis Redlock), order summary, contract generation, booking state machine	A user can search, select, and create bookings for an event
S7–S8	13–16	Payments & Escrow	Payment gateway integration, outbox pattern, escrow module, refund policy engine (configurable), payment ledger, idempotency, webhook handling	End-to-end payment + escrow for a booking works reliably
S9	17–18	Event Execution	Vendor check-in PWA, real-time SignalR dashboard, event day timeline, SLA monitoring, auto-escalation Durable Function, deliverable upload	Vendor can check in and users see live status
S10	19–20	Post-Event	Dispute module (all states), admin arbitration portal, escrow release, settlement + payout, commission config, tax calculation, invoice PDF	Full post-event flow works including dispute resolution
S11	21–22	Reviews & Admin	Multi-dimension review, moderation queue, vendor response, rating → search rank feed, admin dashboard (all	Reviews live, admin can manage full platform

			12 capabilities), fraud detection integration	
S12	23–24	Hardening & Launch	Security pen test, performance testing (k6), E2E Playwright suite, load test (500 concurrent), OWASP ZAP, accessibility audit, SLO dashboards, prod deploy, go-live runbook	Production deploy with SLO dashboards live

12. Non-Functional Requirements (SLOs)

Requirement	Target	Measurement	Alert Threshold
API Availability	99.9% monthly uptime	Azure Monitor synthetic tests every 1 min	< 99.5% in any 24h window
API Response Time (p95)	< 400ms for reads, < 800ms for writes	App Insights percentile metrics	p95 > 1000ms for > 5 min
Payment Webhook Processing	< 30 seconds end-to-end	Custom metric from webhook receipt to escrow update	> 60 seconds on any webhook
Search Response	< 200ms (p95)	Azure AI Search latency + Redis cache hit rate	p95 > 500ms or cache hit < 80%
Slot Lock Acquisition	< 100ms (p95)	Redis operation latency metric	> 300ms or lock failure rate > 1%
Notification Delivery	< 60 seconds after trigger event	Notification Hub delivery reports	> 5% failure rate on any channel
DB Write Throughput	500 concurrent writes/second	SQL DTU / vCore metrics	> 80% DTU sustained for > 10 min
Error Rate	< 0.1% of all API calls (5xx)	App Insights failure rate	> 1% in any 5-minute window
Data Recovery (RTO)	< 4 hours	Defined in DR runbook, tested quarterly	DR drill failure
Data Backup (RPO)	< 1 hour data loss	Azure SQL automated backups every 12h + log shipping	Backup failure alert

12.1 Observability Setup

Azure Monitor Stack

- **App Insights SDK:** Instrument every .NET 10 API and Angular app for distributed tracing, custom events, and dependency tracking
- **Log Analytics Workspace:** Centralised log aggregation from all services with KQL query dashboards for operations team
- **SLO Dashboard (Power BI / Grafana):** Real-time dashboard showing: booking conversion rate, payment success rate, vendor SLA compliance, average dispute resolution time
- **PagerDuty Integration:** Azure Monitor alert rules → PagerDuty → on-call rotation for P0 incidents (payment failures, DB downtime, error rate spikes)
- **Synthetic Tests:** Azure Monitor availability tests hitting /health endpoint and a critical booking path every minute from 3 Azure regions

13. Developer Environment Setup

13.1 Prerequisites

Tool	Version	Purpose	Install
.NET SDK	10.x	Backend development	dotnet.microsoft.com
Node.js	22 LTS	Angular frontend	nodejs.org
Angular CLI	20.x	Angular development	npm install -g @angular/cli@20
Docker Desktop	Latest	Testcontainers, local SQL/Redis	docker.com
Azure CLI	2.x	Azure resource management	docs.microsoft.com/cli
Azure Bicep	0.x	IaC deployment	az bicep install
VS Code	Latest	Recommended IDE	code.visualstudio.com
Azure Data Studio	Latest	SQL management	Optional — for DB work

13.2 Local Setup Steps

1. Clone the monorepo: git clone https://dev.azure.com/org/event-planner.git

2. Copy .env.template to .env and fill in Azure AD B2C tenant details (from team password manager)
3. Start local dependencies: docker compose up -d (starts SQL Server 2022 + Redis + Azurite)
4. Run EF Core migrations: cd src/backend && dotnet ef database update --project EventPlanner.Infrastructure
5. Seed development data: dotnet run --project scripts/Seed -- --env dev
6. Start the backend API: dotnet run --project src/EventPlanner.Api (runs on http://localhost:5001)
7. Start the user portal: cd src/frontend && ng serve user-portal (runs on http://localhost:4200)
8. Start the vendor portal: ng serve vendor-portal (runs on http://localhost:4201)
9. Start the admin portal: ng serve admin-portal (runs on http://localhost:4202)

13.3 docker-compose.yml (Local Dependencies)

```
services:
```

```
  sql:
```

```
    image: mcr.microsoft.com/mssql/server:2022-latest
```

```
    environment:
```

```
      SA_PASSWORD: "EventPlanner@Local123"
```

```
      ACCEPT_EULA: "Y"
```

```
    ports: ["1433:1433"]
```

```
    volumes: [sql-data:/var/opt/mssql]
```

```
  redis:
```

```
    image: redis:7-alpine
```

```
    ports: ["6379:6379"]
```

```
    command: redis-server --requirepass "localredispass"
```

```
  azurite:           # Azure Storage emulator
```

```
image: mcr.microsoft.com/azure-storage/azurite
```

```
ports: ["10000:10000", "10001:10001", "10002:10002"]
```

```
elasticsearch:      # Local search (replaces Azure AI Search in dev)
```

```
image: elasticsearch:8.12.0
```

```
environment: { discovery.type: single-node, xpack.security.enabled: "false" }
```

```
ports: ["9200:9200"]
```

```
volumes:
```

```
sql-data:
```

14. Architecture Decision Records (ADRs)

The following decisions have been made and documented. All future architectural decisions must follow the ADR format and be stored in `/docs/adr/`.

ADR #	Decision	Rationale	Trade-offs Accepted
ADR-001	Modular Monolith over Microservices	Team size < 10. Operational complexity of microservices outweighs benefits at launch. Module boundaries enforced by ArchUnit.	Single deployment unit. Extract to services when team/load demands.
ADR-002	Vertical Slice Architecture	Keeps all code for a feature co-located. Eliminates cross-layer navigation. Easier onboarding.	Some code duplication across slices (acceptable — prefer duplication over wrong abstraction).
ADR-003	Outbox Pattern for all async messages	Guarantees at-least-once delivery of domain events. Prevents lost events on app crash mid-transaction.	Extra outbox table and background worker. Slight processing latency (~5s).
ADR-004	Redis Redlock for slot locking	Prevents double-booking under concurrent load. 15-minute TTL auto-releases abandoned locks.	Redis is now a critical dependency for bookings. Must be highly available (cluster mode).
ADR-005	Cosmos DB for audit log only	Immutable, append-only audit trail benefits from Cosmos schema flexibility and Change Feed.	Adds Cosmos operational complexity. Justified by compliance and dispute evidence needs.
ADR-006	Cursor-based pagination	Offset pagination degrades at scale (OFFSET 10000 is slow). Cursor is O(log n).	Cannot jump to arbitrary pages. UI must use infinite scroll or next/prev only.
ADR-007	Angular Signals over NgRx	Angular 20 Signals API provides built-in reactivity without NgRx boilerplate for most use cases.	Use NgRx Signal Store for complex cross-component state only (booking cart).
ADR-008	Azure AD B2C for all auth	Managed auth removes custom token logic. Supports OTP, SSO, MFA out of the box.	Vendor lock-in to Azure AD B2C. Migration cost if switching providers.

15. Definition of Done

Every user story must meet ALL of the following criteria before it can be marked Done:

- Code reviewed by at least 1 peer (2 for payment/auth changes)
- Unit tests written and passing ($\geq 90\%$ coverage on new code)
- Integration tests added for new API endpoints
- Architecture boundary tests still passing (ArchUnit)
- No new SonarCloud critical or blocker issues
- API endpoint documented in OpenAPI spec
- Database migration is idempotent and has rollback script
- Feature deployed to dev environment and smoke-tested
- Acceptance criteria from ticket verified by QA
- New Azure resources added to Bicep templates
- No secrets committed to repository (pre-commit hook enforced)
- PII fields masked in all log statements
- Responsive UI tested on mobile viewport (vendor PWA)
- Ticket updated with deployment notes and test evidence

EVENT PLANNER PLATFORM

Full-Stack Development Blueprint

Angular 20	.NET Core 10	Azure Cloud	v1.0 Feb 2026
------------	--------------	-------------	---------------

Stack Layer	Technology	Version	Purpose
Frontend	Angular	20 (Signals + SSR)	User & Vendor Web Applications
Backend	.NET Core	10 (Minimal API)	REST APIs, CQRS, Domain Logic
Database	Azure SQL + Cosmos DB	Latest	Relational + Audit/Event Store
Cache	Azure Redis Enterprise	7.x	Slot Locking, Search Cache, Sessions
Search	Azure AI Search	Latest	Vendor & Service Discovery
Auth	Azure AD B2C	Latest	JWT, OTP, RBAC, Social SSO
Messaging	Azure Service Bus	Premium	Async Events, Dead-letter Queue
Realtime	Azure SignalR Service	Latest	Live Dashboard, Chat, Notifications
Serverless	Azure Durable Functions	v4	Timers, SLA, Workflows
Storage	Azure Blob + CDN	Latest	Portfolios, Contracts, Photos
DevOps	Azure DevOps	Latest	CI/CD, Boards, Repos, Test Plans

Confidential — Engineering Team Reference

1. Solution Architecture Overview

The Event Planner Platform is designed as a Modular Monolith on the backend and a standalone Angular 20 application on the frontend. The architecture prioritises developer velocity in early phases while enforcing clean domain boundaries that allow individual modules to be extracted into independent microservices when scaling pressure demands it.

1.1 Architecture Style



Backend: Modular Monolith with Vertical Slice Architecture

- **Single deployable .NET 10 application:** contains independently testable feature slices, each owning its own request, handler, validator, and repository
- **Domain modules are strictly isolated:** no direct cross-module references — all communication via MediatR in-process events or Azure Service Bus for async
- **Extraction-ready:** any module (e.g. Payment, Dispute) can be extracted to a standalone Azure Container App with a Service Bus interface change only
- **CQRS pattern enforced:** Commands mutate state; Queries are read-only and can bypass the write model to read replicas



Frontend: Angular 20 Standalone Architecture

- **Signal-based state management:** no NgRx store overhead for simple state; use Angular Signals for reactivity throughout
- **Feature-based folder structure:** each domain feature (bookings, vendors, events) is a standalone module with lazy-loaded routes
- **SSR with Angular Universal:** for SEO-sensitive pages (vendor profiles, service listings) using Angular 20 SSR
- **PWA for Vendor Mobile App:** vendors use a Progressive Web App for field check-in, deliverable upload, and earnings dashboard

1.2 System Topology

Layer	Component	Hosting	Scale Strategy
Edge	Azure Front Door + WAF	Azure Global	Geo-routing, DDoS, TLS termination
API Gateway	Azure API Management (APIM)	Azure Region	Rate limiting, JWT validation, versioning
Frontend	Angular 20 SPA + SSR	Azure Static Web Apps	CDN edge, auto-scale
Backend API	.NET 10 Minimal API	Azure Container Apps	Horizontal scale, min 2 replicas
Background Jobs	Azure Durable Functions	Consumption / Premium	Event-driven, timer-triggered
Primary DB	Azure SQL Hyperscale	Zone-redundant	Read replicas for queries
Audit/Events	Azure Cosmos DB	Multi-region	Change Feed for projections
Cache	Azure Redis Enterprise	Zone-redundant	Cluster mode, Redlock
Search	Azure AI Search (Standard)	Azure Region	Index replicas for throughput
Async Messaging	Azure Service Bus Premium	Zone-redundant	Partitioned topics, DLQ
File Storage	Azure Blob Storage (LRS)	Azure Region	CDN for media delivery
Realtime	Azure SignalR Service	Standard tier	Hub-based connections
Notifications	Azure Communication Services	Azure Global	Email, SMS, Push, WhatsApp

Monitoring	Azure Monitor + App Insights	Azure Region	Log Analytics workspace
------------	------------------------------	--------------	-------------------------

1.3 Module Map

The backend is organised into the following bounded-context modules. Each module owns its own tables, API endpoints, and domain logic:

Module	Responsibility	Key Entities	Publishes Events
Identity	Auth, registration, RBAC, OTP	User, Vendor, AdminUser, Role	UserRegistered, VendorVerified
Catalogue	Service categories, services, sub-services, pricing	ServiceCategory, Service, SubService, PriceRule	ServicePublished, PriceUpdated
Availability	Vendor calendars, slot locking, conflict detection	AvailabilitySlot, SlotLock	SlotLocked, SlotReleased
Events	Parent event entity, budget tracking, timeline	Event, EventTimeline, BudgetLine	EventCreated, EventCompleted
Bookings	Full booking lifecycle, state machine, contracts	VendorBooking, BookingAuditLog, Contract	BookingConfirmed, BookingCancelled
Payments	Payment initiation, webhooks, escrow, ledger	Payment, PaymentLedger, EscrowAccount, Outbox	PaymentReceived, EscrowReleased
Refunds	Policy engine, refund calculation, payout	RefundPolicy, RefundRequest	RefundInitiated, RefundCompleted
Disputes	Dispute lifecycle, evidence, admin arbitration	Dispute, DisputeEvidence, DisputeResolution	DisputeRaised, DisputeResolved
Settlement	Commission, tax, vendor payout, invoicing	Settlement, CommissionConfig, TaxRule, Invoice	VendorPaid, InvoiceGenerated
Reviews	Multi-dim ratings, moderation, vendor response	Review, ReviewDimension, ReviewResponse	ReviewPublished
Messaging	Thread-per-booking chat, notifications, prefs	MessageThread, Message, NotificationPref	MessageSent

Analytics	Platform KPIs, vendor performance, fraud flags	BookingMetric, VendorScore, FraudFlag	MetricRecorded
Admin	Admin portal aggregates all modules	AdminAction, AuditEntry	AdminActionTaken

2. Repository & Project Structure

Use a single Git monorepo managed with Azure DevOps. The monorepo keeps frontend and backend in one repository for atomic commits, shared type contracts, and unified CI/CD pipelines.

2.1 Monorepo Layout

```
event-planner/                                # Root monorepo

├─ .azure/                                    # Azure Bicep IaC templates

|   └─ main.bicep

|   └─ modules/                              # sql.bicep, redis.bicep, servicebus.bicep...

|   └─ environments/                        # dev.bicepparam, staging.bicepparam, prod.bicepparam

├─ src/

|   └─ backend/                              # .NET Core 10 Solution

|       └─ EventPlanner.sln

|       └─ src/

|           └─ EventPlanner.Api/             # ASP.NET Minimal API host

|           └─ EventPlanner.Domain/          # Entities, Value Objects, Domain Events

|           └─ EventPlanner.Application/     # CQRS Commands, Queries, Handlers

|           └─ EventPlanner.Infrastructure/  # EF Core, Redis, Azure SDK, Outbox

|           └─ EventPlanner.Modules.Identity/

|           └─ EventPlanner.Modules.Catalogue/
```

```
| | | └─ EventPlanner.Modules.Availability/

| | | └─ EventPlanner.Modules.Events/

| | | └─ EventPlanner.Modules.Bookings/

| | | └─ EventPlanner.Modules.Payments/

| | | └─ EventPlanner.Modules.Refunds/

| | | └─ EventPlanner.Modules.Disputes/

| | | └─ EventPlanner.Modules.Settlement/

| | | └─ EventPlanner.Modules.Reviews/

| | | └─ EventPlanner.Modules.Messaging/

| | | └─ EventPlanner.Modules.Analytics/

| | └─ tests/

| | └─ EventPlanner.UnitTests/

| | └─ EventPlanner.IntegrationTests/

| | └─ EventPlanner.ArchTests/ # ArchUnitNET boundary enforcement

└─ frontend/ # Angular 20 Workspace
```


2.2 Backend Module Internal Structure

Every module follows the same Vertical Slice folder convention. No exceptions. This keeps the codebase consistent and onboarding fast:

EventPlanner.Modules.Bookings/

└─ Features/

| └─ CreateBooking/

| | └─ CreateBookingCommand.cs # MediatR IRequest<Result<BookingId>>

| | └─ CreateBookingHandler.cs # IRequestHandler - all logic here

| | └─ CreateBookingValidator.cs # FluentValidation rules

| | └─ CreateBookingEndpoint.cs # Minimal API endpoint registration

| | └─ CreateBookingResponse.cs # Response DTO

| └─ GetBooking/

| └─ UpdateBookingStatus/

| └─ CancelBooking/

| └─ ListBookings/

└─ Domain/

| └─ Booking.cs # Aggregate root with state machine

| └─ BookingStatus.cs # Enum + transition guard methods

```
| | └─ BookingAuditEntry.cs                # Domain event for audit log

| | └─ Events/                             # Domain events: BookingConfirmed, etc.

└─ Infrastructure/

| | └─ BookingRepository.cs

| | └─ BookingDbContext.cs                # Module-scoped DbContext

| | └─ BookingConfiguration.cs            # EF Core Fluent API config

└─ BookingsModule.cs                      # IHostedService registration
```

3. Database Design

Azure SQL Hyperscale is the primary OLTP store. Azure Cosmos DB (NoSQL) is used exclusively for the immutable audit log and event sourcing projections. Redis is used for caching and distributed locking — never as a primary store.

3.1 Core Table Schemas

users

```

CREATE TABLE users (

    user_id          UNIQUEIDENTIFIER  DEFAULT NEWSEQUENTIALID() PRIMARY KEY,

    full_name        NVARCHAR(150)     NOT NULL,

    email            NVARCHAR(255)     NOT NULL  UNIQUE,

    phone            NVARCHAR(20)       NOT NULL  UNIQUE,

    password_hash    NVARCHAR(500)     NULL,      -- NULL if SSO only

    city             NVARCHAR(100)     NULL,

    status           NVARCHAR(20)       NOT NULL  DEFAULT 'ACTIVE',  -- ACTIVE | SUSPENDED | DELETED

    created_at       DATETIMEOFFSET    NOT NULL  DEFAULT SYSDATETIMEOFFSET(),

    updated_at       DATETIMEOFFSET    NOT NULL  DEFAULT SYSDATETIMEOFFSET()

);

CREATE INDEX IX_users_email ON users(email);

CREATE INDEX IX_users_phone ON users(phone);

```

vendors

```
CREATE TABLE vendors (  
  
    vendor_id          UNIQUEIDENTIFIER  DEFAULT NEWSEQUENTIALID() PRIMARY KEY,  
  
    user_id            UNIQUEIDENTIFIER  NOT NULL REFERENCES users(user_id),  
  
    business_name       NVARCHAR(200)    NOT NULL,  
  
    gstin              NVARCHAR(20)      NULL,  
  
    kyc_status          NVARCHAR(20)     NOT NULL DEFAULT 'PENDING', --  
PENDING|UNDER_REVIEW|VERIFIED|REJECTED  
  
    insurance_url       NVARCHAR(500)     NULL,  
  
    insurance_expiry    DATE              NULL,  
  
    avg_rating          DECIMAL(3,2)     NOT NULL DEFAULT 0,  
  
    total_reviews       INT              NOT NULL DEFAULT 0,  
  
    is_published        BIT              NOT NULL DEFAULT 0,  
  
    created_at          DATETIMEOFFSET   NOT NULL DEFAULT SYSDATETIMEOFFSET(),  
  
    updated_at          DATETIMEOFFSET   NOT NULL DEFAULT SYSDATETIMEOFFSET()  
  
);
```

service_categories / services / sub_services


```

pricing_type      NVARCHAR(20)      NOT NULL,    -- HOURLY | FLAT | PACKAGE

min_hours         INT                NULL,

is_active         BIT                NOT NULL    DEFAULT 1

);

CREATE TABLE sub_services (          -- NEW: Sub-service hierarchy

sub_service_id    UNIQUEIDENTIFIER  DEFAULT NEWSEQUENTIALID() PRIMARY KEY,

service_id        UNIQUEIDENTIFIER  NOT NULL    REFERENCES services(service_id),

name              NVARCHAR(200)     NOT NULL,    -- Drone Shoot, Live Stream, Album...

additional_price   DECIMAL(12,2)     NOT NULL    DEFAULT 0,

bundle_discount   DECIMAL(5,2)      NULL,        -- % discount when booked with parent

requires equip    BIT                NOT NULL    DEFAULT 0,

equipment_list     NVARCHAR(1000)    NULL,

lead_time_hours   INT                NOT NULL    DEFAULT 0,

is_active         BIT                NOT NULL    DEFAULT 1

```

);

events (Parent Entity — NEW)

```

CREATE TABLE events (
    -- NEW: Parent record for all vendor bookings

    event_id          UNIQUEIDENTIFIER  DEFAULT NEWSEQUENTIALID() PRIMARY KEY,

    user_id           UNIQUEIDENTIFIER  NOT NULL  REFERENCES users(user_id),

    event_name         NVARCHAR(200)     NOT NULL,

    event_type         NVARCHAR(50)      NOT NULL,    -- WEDDING|BIRTHDAY|CORPORATE|RECEPTION

    event_date         DATE               NOT NULL,

    city               NVARCHAR(100)     NOT NULL,

    venue_details      NVARCHAR(500)     NULL,

    total_budget       DECIMAL(14,2)     NULL,

    committed_amount   DECIMAL(14,2)     NOT NULL  DEFAULT 0,

    status             NVARCHAR(20)      NOT NULL  DEFAULT 'DRAFT',

    created_at         DATETIMEOFFSET    NOT NULL  DEFAULT SYSDATETIMEOFFSET()

);

```

vendor_bookings

```
CREATE TABLE vendor_bookings (  
  
    booking_id          UNIQUEIDENTIFIER  DEFAULT NEWSEQUENTIALID() PRIMARY KEY,  
  
    event_id            UNIQUEIDENTIFIER  NOT NULL REFERENCES events(event_id),    -- Parent  
  
    user_id             UNIQUEIDENTIFIER  NOT NULL REFERENCES users(user_id),  
  
    vendor_id           UNIQUEIDENTIFIER  NOT NULL REFERENCES vendors(vendor_id),  
  
    service_id          UNIQUEIDENTIFIER  NOT NULL REFERENCES services(service_id),  
  
    status              NVARCHAR(30)      NOT NULL DEFAULT 'DRAFT',  
  
    -- Status values: DRAFT|REQUESTED|VENDOR_ACCEPTED|CONFIRMED|IN_PROGRESS|  
  
    --                  COMPLETED|SETTLED|DISPUTED|CANCEL_REQUESTED|CANCELLED|  
  
    --                  REFUND_INITIATED|REFUND_COMPLETED|EXPIRED  
  
    total_amount         DECIMAL(14,2)    NOT NULL,  
  
    advance_amount       DECIMAL(14,2)    NOT NULL,  
  
    advance_paid_at      DATETIMEOFFSET   NULL,  
  
    contract_url         NVARCHAR(500)    NULL,  
  
    idempotency_key      UNIQUEIDENTIFIER  NOT NULL DEFAULT NEWID(),
```

```

created_at          DATETIMEOFFSET      NOT NULL   DEFAULT SYSDATETIMEOFFSET(),

updated_at          DATETIMEOFFSET      NOT NULL   DEFAULT SYSDATETIMEOFFSET()

);

CREATE TABLE booking_sub_services (    -- Many-to-many: booking ↔ sub_service

booking_id          UNIQUEIDENTIFIER    NOT NULL   REFERENCES vendor_bookings(booking_id),

sub_service_id      UNIQUEIDENTIFIER    NOT NULL   REFERENCES sub_services(sub_service_id),

price_at_booking    DECIMAL(12,2)      NOT NULL,

PRIMARY KEY (booking_id, sub_service_id)

);

```

payments & escrow

```

CREATE TABLE payment_ledger (

    ledger_id          UNIQUEIDENTIFIER  DEFAULT NEWSEQUENTIALID() PRIMARY KEY,

    booking_id         UNIQUEIDENTIFIER  NOT NULL REFERENCES vendor_bookings(booking_id),

    payment_type        NVARCHAR(20)      NOT NULL,    -- ADVANCE | FINAL | REFUND

    amount             DECIMAL(14,2)      NOT NULL,

    currency            CHAR(3)           NOT NULL     DEFAULT 'INR',

    gateway             NVARCHAR(50)      NOT NULL,    -- RAZORPAY | STRIPE | CASHFREE

    gateway_ref         NVARCHAR(200)     NULL,

    idempotency_key     NVARCHAR(200)     NOT NULL,

    status              NVARCHAR(20)      NOT NULL,    -- PENDING|SUCCESS|FAILED|REFUNDED

    created_at          DATETIMEOFFSET    NOT NULL     DEFAULT SYSDATETIMEOFFSET()

);

```

```

CREATE TABLE escrow_accounts (

    escrow_id          UNIQUEIDENTIFIER  DEFAULT NEWSEQUENTIALID() PRIMARY KEY,

```

```

booking_id          UNIQUEIDENTIFIER NOT NULL REFERENCES vendor_bookings(booking_id),

held_amount         DECIMAL(14,2)     NOT NULL,

platform_fee        DECIMAL(14,2)     NULL,

tax_amount          DECIMAL(14,2)     NULL,

damage_deduction    DECIMAL(14,2)     NOT NULL DEFAULT 0,

vendor_payable       DECIMAL(14,2)     NULL,

released_at         DATETIMEOFFSET    NULL,

released_to         NVARCHAR(10)      NULL          -- VENDOR | USER

);

```

```

CREATE TABLE outbox_messages (          -- Outbox Pattern for reliable messaging

message_id          UNIQUEIDENTIFIER DEFAULT NEWSEQUENTIALID() PRIMARY KEY,

correlation_id      NVARCHAR(200)     NOT NULL,

message_type        NVARCHAR(200)     NOT NULL,

payload             NVARCHAR(MAX)     NOT NULL,    -- JSON

```

```
created_at      DATETIMEOFFSET      NOT NULL  DEFAULT SYSDATETIMEOFFSET(),

processed_at    DATETIMEOFFSET      NULL,

retry_count     INT                NOT NULL  DEFAULT 0

);

CREATE INDEX IX_outbox_unprocessed ON outbox_messages(processed_at) WHERE processed_at IS NULL;
```

disputes & reviews

```

CREATE TABLE disputes (

    dispute_id          UNIQUEIDENTIFIER  DEFAULT NEWSEQUENTIALID() PRIMARY KEY,

    booking_id          UNIQUEIDENTIFIER  NOT NULL  REFERENCES vendor_bookings(booking_id),

    raised_by_user_id   UNIQUEIDENTIFIER  NOT NULL  REFERENCES users(user_id),

    dispute_type         NVARCHAR(30)      NOT NULL,    -- QUALITY|NO_SHOW|DAMAGE|PARTIAL

    description          NVARCHAR(2000)    NOT NULL,

    status               NVARCHAR(20)      NOT NULL  DEFAULT 'RAISED',

    -- RAISED|EVIDENCE_SUBMITTED|ADMIN_REVIEW|RESOLVED|ESCALATED

    resolution          NVARCHAR(20)      NULL,        -- FULL_REFUND|PARTIAL_REFUND|DISMISSED

    resolved_by          UNIQUEIDENTIFIER  NULL,        -- AdminUser FK

    resolved_at          DATETIMEOFFSET    NULL,

    created_at           DATETIMEOFFSET    NOT NULL  DEFAULT SYSDATETIMEOFFSET()

);

```

```

CREATE TABLE reviews (

```

```
review_id          UNIQUEIDENTIFIER  DEFAULT NEWSEQUENTIALID() PRIMARY KEY,

booking_id         UNIQUEIDENTIFIER  NOT NULL  REFERENCES vendor_bookings(booking_id),

user_id            UNIQUEIDENTIFIER  NOT NULL  REFERENCES users(user_id),

vendor_id          UNIQUEIDENTIFIER  NOT NULL  REFERENCES vendors(vendor_id),

overall_rating     TINYINT           NOT NULL  CHECK (overall_rating BETWEEN 1 AND 5),

punctuality        TINYINT           NULL      CHECK (punctuality BETWEEN 1 AND 5),

quality            TINYINT           NULL      CHECK (quality BETWEEN 1 AND 5),

communication       TINYINT           NULL      CHECK (communication BETWEEN 1 AND 5),

value_for_money     TINYINT           NULL      CHECK (value_for_money BETWEEN 1 AND 5),

comment            NVARCHAR(2000)     NULL,

vendor_response     NVARCHAR(1000)    NULL,

moderation_status  NVARCHAR(20)       NOT NULL  DEFAULT 'PENDING',

is_published        BIT               NOT NULL  DEFAULT 0,

created_at          DATETIMEOFFSET    NOT NULL  DEFAULT SYSDATETIMEOFFSET()
```

);

3.2 Cosmos DB — Audit Log Schema

Every booking status transition is written immutably to Cosmos DB. This is the compliance and dispute evidence store. Partition key: /bookingId.

}

4. API Design Standards

All APIs follow REST conventions with versioned routes, consistent error envelopes, cursor-based pagination, and idempotency keys on mutating operations. The OpenAPI spec in `/contracts/openapi.yaml` is the single source of truth — generated from .NET source attributes.

4.1 URL & Versioning Conventions

Base URL pattern

https://api.eventplanner.com/api/v1/{resource}

Resource patterns

GET	/api/v1/vendors	List vendors (paginated)
-----	-----------------	--------------------------

GET	/api/v1/vendors/{id}	Get vendor profile
-----	----------------------	--------------------

POST	/api/v1/vendors	Register vendor
------	-----------------	-----------------

PATCH	/api/v1/vendors/{id}/kyc	Update KYC status (Admin)
-------	--------------------------	---------------------------

GET	/api/v1/events	List user events
-----	----------------	------------------

POST	/api/v1/events	Create event (parent)
------	----------------	-----------------------

GET	/api/v1/events/{id}	Get event + all bookings
-----	---------------------	--------------------------

POST	/api/v1/bookings	Create booking (draft)
------	------------------	------------------------

PATCH	/api/v1/bookings/{id}/status	Transition state
-------	------------------------------	------------------

POST	/api/v1/bookings/{id}/cancel	Cancel with reason
POST	/api/v1/payments/initiate	Initiate payment
POST	/api/v1/payments/webhook	Gateway webhook (unsigned)
POST	/api/v1/disputes	Raise dispute
PATCH	/api/v1/disputes/{id}/resolve	Admin resolution
POST	/api/v1/reviews	Submit review
PATCH	/api/v1/reviews/{id}/respond	Vendor response
GET	/api/v1/search/vendors	Search vendors
GET	/api/v1/search/services	Search services + sub-services

4.2 Standard Response Envelope

```
// SUCCESS RESPONSE
```

```
{  
  
  "success": true,  
  
  "data": { ... },           // Actual payload  
  
  "meta": {  
  
    "requestId": "trace-id",  // X-Request-ID header echoed  
  
    "timestamp": "2026-02-20T14:30:00Z"  
  
  }  
  
}
```

```
// PAGINATED RESPONSE
```

```
{  
  
  "success": true,  
  
  "data": [ ... ],  
  
  "pagination": {
```

```
"cursor": "base64-encoded-cursor", // Cursor-based, NOT offset

"hasMore": true,

"pageSize": 20

}

}

// ERROR RESPONSE

{

"success": false,

"error": {

"code": "BOOKING_SLOT_UNAVAILABLE",

"message": "The selected time slot is no longer available.",

"details": [ { "field": "slotId", "issue": "Slot expired" } ],

"requestId": "trace-id"

}
```

```
}
```

4.3 Idempotency

All POST endpoints that create bookings or initiate payments MUST require an Idempotency-Key header. The server stores the response for 24 hours and returns the same response for duplicate requests with the same key.

```
POST /api/v1/payments/initiate
```

```
Headers:
```

```
Authorization: Bearer {jwt}
```

```
Idempotency-Key: {uuid-v4}           // Client-generated, unique per attempt
```

```
Content-Type: application/json
```

```
// .NET Handler checks outbox/idempotency table before processing
```

```
// If key exists: return cached response (202 Accepted)
```

```
// If key new: process and store response in idempotency table
```


5. Backend Implementation Guide (.NET Core 10)

5.1 Project Setup & Key Packages

```
# Create solution
```

```
dotnet new sln -n EventPlanner
```

```
dotnet new webapi -n EventPlanner.Api --use-minimal-apis
```

```
dotnet new classlib -n EventPlanner.Domain
```

```
dotnet new classlib -n EventPlanner.Application
```

```
dotnet new classlib -n EventPlanner.Infrastructure
```

```
# Essential NuGet packages
```

```
dotnet add package MediatR # CQRS dispatcher
```

```
dotnet add package FluentValidation.AspNetCore # Request validation
```

```
dotnet add package Microsoft.EntityFrameworkCore # ORM
```

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
```

```
dotnet add package MassTransit.AzureServiceBus # Messaging
```

```
dotnet add package StackExchange.Redis # Redis client
```

```
dotnet add package Azure.Storage.Blobs # Blob storage
```

```
dotnet add package Azure.Search.Documents           # AI Search client

dotnet add package Azure.Security.KeyVault.Secrets

dotnet add package Azure.Identity                   # Managed Identity

dotnet add package Polly.Extensions.Http            # Resilience policies

dotnet add package Serilog.AspNetCore               # Structured logging

dotnet add package ArchUnitNET                      # Architecture tests

dotnet add package Bogus                           # Test data generation

dotnet add package Microsoft.AspNetCore.SignalR    # Real-time hub
```

5.2 Vertical Slice — Complete Example (Create Booking)

CreateBookingCommand.cs

```
using MediatR;

using EventPlanner.Domain.Bookings;

namespace EventPlanner.Modules.Bookings.Features.CreateBooking;

public record CreateBookingCommand(

    Guid EventId,

    Guid VendorId,

    Guid ServiceId,

    IReadOnlyList<Guid> SubServiceIds,

    DateOnly EventDate,

    TimeOnly StartTime,

    TimeOnly EndTime

) : IRequest<Result<BookingId>>;
```



```
using FluentValidation;

public class CreateBookingValidator : AbstractValidator<CreateBookingCommand>
{
    public CreateBookingValidator()
    {
        RuleFor(x => x.EventId).NotEmpty();

        RuleFor(x => x.VendorId).NotEmpty();

        RuleFor(x => x.ServiceId).NotEmpty();

        RuleFor(x => x.EventDate).GreaterThan(DateOnly.FromDateTime(DateTime.UtcNow))

            .WithMessage("Event date must be in the future.");

        RuleFor(x => x.StartTime).LessThan(x => x.EndTime)

            .WithMessage("Start time must be before end time.");
    }
}
```



CreateBookingHandler.cs

```
public class CreateBookingHandler : IRequestHandler<CreateBookingCommand, Result<BookingId>>

{

    private readonly IBookingRepository _bookings;

    private readonly IAvailabilityService _availability; // Cross-module via interface

    private readonly IPricingService _pricing;

    private readonly IOutboxService _outbox;

    private readonly IAuditLogService _audit;

    public async Task<Result<BookingId>> Handle(

        CreateBookingCommand cmd, CancellationToken ct)

    {

        // 1. Verify slot is available (calls Availability module)

        var lockResult = await _availability.TryAcquireSlotLockAsync(

            cmd.VendorId, cmd.EventDate, cmd.StartTime, cmd.EndTime, ct);

        if (!lockResult.IsSuccess) return Result.Fail("Slot no longer available");
```

```
// 2. Calculate pricing

var price = await _pricing.CalculateTotalAsync(

    cmd.ServiceId, cmd.SubServiceIds, ct);

// 3. Create aggregate root

var booking = Booking.Create(

    cmd.EventId, cmd.VendorId, cmd.ServiceId,

    cmd.SubServiceIds, price, cmd.EventDate);

// 4. Persist + publish outbox event atomically

await _bookings.AddAsync(booking, ct);

await _outbox.PublishAsync(new BookingCreatedEvent(booking.Id), ct);

// 5. Audit log (Cosmos DB)
```

```
        await _audit.RecordAsync(new AuditEntry(

            booking.Id, "CREATED", null, "DRAFT", Actor.FromContext()), ct);

    }

    return Result.Ok(booking.Id);

}

}
```

CreateBookingEndpoint.cs (Minimal API)

```
public static class CreateBookingEndpoint

{

    public static void MapCreateBooking(this IEndpointRouteBuilder app)

    {

        app.MapPost("/api/v1/bookings", async (

            CreateBookingRequest req,

            IMediator mediator,

            HttpContext ctx,

            CancellationToken ct) =>

        {

            var command = new CreateBookingCommand(

                req.EventId, req.VendorId, req.ServiceId,

                req.SubServiceIds, req.EventDate, req.StartTime, req.EndTime);

            var result = await mediator.Send(command, ct);
```

```

        return result.IsSuccess

        ? Results.Created($"/api/v1/bookings/{result.Value}", new { bookingId =
result.Value })

        : Results.BadRequest(ApiError.From(result));

    })

    .RequireAuthorization("UserRole")

    .AddEndpointFilter<IdempotencyFilter>()

    .AddEndpointFilter<ValidationFilter<CreateBookingRequest>>()

    .WithName("CreateBooking")

    .WithOpenApi();

}

}

```

5.3 Booking State Machine (Domain Aggregate)

```
public class Booking : AggregateRoot

{

    public BookingStatus Status { get; private set; }

    public Result TransitionTo(BookingStatus newStatus, Actor actor)

    {

        if (!AllowedTransitions[Status].Contains(newStatus))

            return Result.Fail($"Cannot move from {Status} to {newStatus}");

        var oldStatus = Status;

        Status = newStatus;

        UpdatedAt = DateTimeOffset.UtcNow;

        AddDomainEvent(new BookingStatusChangedEvent(

            Id, oldStatus, newStatus, actor, DateTimeOffset.UtcNow));
    }
}
```

```
return Result.Ok();

}

private static readonly Dictionary<BookingStatus, BookingStatus[]> AllowedTransitions = new()

{

    [BookingStatus.Draft] = [BookingStatus.Requested],

    [BookingStatus.Requested] = [BookingStatus.VendorAccepted, BookingStatus.Expired],

    [BookingStatus.VendorAccepted] = [BookingStatus.Confirmed],

    [BookingStatus.Confirmed] = [BookingStatus.InProgress,
BookingStatus.CancelRequested],

    [BookingStatus.InProgress] = [BookingStatus.Completed],

    [BookingStatus.Completed] = [BookingStatus.Settled, BookingStatus.Disputed],

    [BookingStatus.Disputed] = [BookingStatus.Settled],

    [BookingStatus.CancelRequested] = [BookingStatus.Cancelled],

    [BookingStatus.Cancelled] = [BookingStatus.RefundInitiated],
```

```
        [BookingStatus.RefundInitiated] = [BookingStatus.RefundCompleted],  
  
        };  
  
    }
```

5.4 Outbox Worker (Reliable Messaging)

```
public class OutboxWorker : BackgroundService

{

    protected override async Task ExecuteAsync(CancellationToken ct)

    {

        while (!ct.IsCancellationRequested)

        {

            // Poll unprocessed outbox messages every 5 seconds

            var messages = await _db.OutboxMessages

                .Where(m => m.ProcessedAt == null && m.RetryCount < 5)

                .OrderBy(m => m.CreatedAt)

                .Take(50)

                .ToListAsync(ct);

            foreach (var msg in messages)

            {
```

```
        try {

            await _serviceBus.PublishAsync(msg.MessageType, msg.Payload, ct);

            msg.ProcessedAt = DateTimeOffset.UtcNow;

        } catch {

            msg.RetryCount++;

        }

    }

    await _db.SaveChangesAsync(ct);

    await Task.Delay(TimeSpan.FromSeconds(5), ct);

}

}

}
```

6. Frontend Implementation Guide (Angular 20)

6.1 Angular Project Setup

```
# Install Angular CLI 20
```

```
npm install -g @angular/cli@20
```

```
# Create workspace with three apps
```

```
ng new event-planner --no-create-application
```

```
cd event-planner
```

```
ng generate application user-portal      --routing --style=scss --standalone
```

```
ng generate application vendor-portal    --routing --style=scss --standalone
```

```
ng generate application admin-portal     --routing --style=scss --standalone
```

```
# Generate shared libraries
```

```
ng generate library ui                   # Design system
```

```
ng generate library data-access          # HTTP services
```

```
ng generate library domain               # Shared interfaces & enums
```

```
ng generate library utils                                # Pipes, guards, interceptors

# Key dependencies

npm install @azure/msal-angular @azure/msal-browser    # Azure AD B2C auth

npm install @microsoft/signalr                          # Real-time

npm install @angular/pwa                               # PWA for vendor app

npm install date-fns                                    # Date utilities

npm install echarts ngx-echarts                        # Charts/analytics
```

6.2 Feature Folder Structure (User Portal)

```
apps/user-portal/src/
```

```
├─ app/
```

```
|   ├─ app.config.ts           # Standalone app config (no AppModule)
```

```
|   ├─ app.routes.ts          # Root lazy routes
```

```
|   ├─ core/
```

```
|   |   ├─ interceptors/
```

```
|   |   |   ├─ auth.interceptor.ts    # Attach JWT to every request
```

```
|   |   |   ├─ idempotency.interceptor.ts
```

```
|   |   |   └─ error.interceptor.ts
```

```
|   |   ├─ guards/
```

```
|   |   |   ├─ auth.guard.ts
```

```
|   |   |   └─ role.guard.ts
```

```
|   |   └─ services/
```

```
|   |       └─ notification.service.ts # SignalR connection
```

```
|   └─ features/
```

```
|   └─ auth/          (login, register, otp)

|   └─ events/        (create-event, event-dashboard, budget-tracker)

|   └─ discovery/     (search, vendor-profile, sub-service-selector)

|   └─ bookings/      (order-summary, booking-list, booking-detail)

|   └─ payments/      (payment-gateway, payment-status)

|   └─ disputes/      (raise-dispute, evidence-upload)

|   └─ reviews/       (write-review, view-reviews)

|   └─ messages/      (chat-thread, notifications)
```

6.3 Signal-Based State — Booking Store

```
// libs/data-access/src/lib/booking.store.ts

import { signalStore, withState, withMethods, patchState } from "@ngrx/signals";

import { inject } from "@angular/core";

import { BookingService } from "../booking.service";

type BookingState = {

  bookings: VendorBooking[];

  selected: VendorBooking | null;

  loading: boolean;

  error: string | null;

};

const initialState: BookingState = {

  bookings: [], selected: null, loading: false, error: null

};
```

```
export const BookingStore = signalStore(

  { providedIn: "root" },

  withState(initialState),

  withMethods((store, svc = inject(BookingService)) => ({

    async loadBookings(eventId: string) {

      patchState(store, { loading: true });

      const result = await svc.getByEvent(eventId);

      patchState(store, { bookings: result.data, loading: false });

    },

    async createBooking(cmd: CreateBookingRequest) {

      patchState(store, { loading: true });

      const result = await svc.create(cmd);

      patchState(store, s => ({

        bookings: [...s.bookings, result.data],
```

```
        loading: false

    });

    return result;

}

}))

);
```

6.4 HTTP Service with Idempotency

```
// libs/data-access/src/lib/booking.service.ts

@Injectable({ providedIn: "root" })

export class BookingService {

  private http = inject(HttpClient);

  private base = "/api/v1/bookings";

  create(cmd: CreateBookingRequest): Observable<ApiResponse<{ bookingId: string }>> {

    const idempotencyKey = crypto.randomUUID();

    return this.http.post<ApiResponse<{ bookingId: string }>>(this.base, cmd, {

      headers: { "Idempotency-Key": idempotencyKey }

    });

  }

  updateStatus(id: string, status: BookingStatus): Observable<ApiResponse<void>> {

    return this.http.patch<ApiResponse<void>>(`${this.base}/${id}/status`, { status });

  }

}
```

```
}  
  
}
```

6.5 Real-Time Event Dashboard (SignalR)

```
// core/services/event-hub.service.ts

@Injectable({ providedIn: "root" })

export class EventHubService {

    private connection: HubConnection;

    private bookingUpdates$ = new Subject<BookingStatusUpdate>();

    connect(eventId: string): void {

        this.connection = new HubConnectionBuilder()

            .withUrl(`/hubs/events?eventId=${eventId}`, {

                accessTokenFactory: () => this.auth.getToken()

            })

            .withAutomaticReconnect()

            .build();

        this.connection.on("BookingStatusUpdated", (update: BookingStatusUpdate) => {
```

```
        this.bookingUpdates$.next(update);

    });

    this.connection.on("VendorCheckedIn", (vendorId: string) => {

        // Update event day dashboard in real-time

    });

    this.connection.start();

}

onBookingUpdate(): Observable<BookingStatusUpdate> {

    return this.bookingUpdates$.asObservable();

}

}
```

6.6 Route Configuration (Lazy Loading)

```
// app.routes.ts

export const routes: Routes = [

  { path: "", loadComponent: () => import("../features/home/home.component") },

  {

    path: "events",

    canActivate: [authGuard],

    children: [

      { path: "", loadComponent: () => import("../features/events/event-list.component") },

      { path: "new", loadComponent: () => import("../features/events/create-event.component") },

      { path: ":id", loadComponent: () => import("../features/events/event-dashboard.component") },

    ]

  },

  {

    path: "vendors/:id",

    loadComponent: () => import("../features/discovery/vendor-profile.component")
```

```
    },  
  
    {  
  
      path: "bookings",  
  
      canActivate: [authGuard],  
  
      loadChildren: () => import("./features/bookings/bookings.routes")  
  
    },  
  
    {  
  
      path: "admin",  
  
      canActivate: [authGuard, roleGuard("Admin")],  
  
      loadChildren: () => import("./features/admin/admin.routes")  
  
    },  
  
  ],  
  
];
```

7. Azure Infrastructure as Code (Bicep)

All Azure resources are provisioned using Azure Bicep templates stored in the `.azure/` folder. Never create resources manually in the portal — everything is code-controlled and environment-parameterised.

7.1 Resource Naming Convention

Resource Type	Pattern	Example
Resource Group	rg-{app}-{env}	rg-eventplanner-prod
App Service / Container App	ca-{app}-api-{env}	ca-eventplanner-api-prod
Azure SQL Server	sql-{app}-{env}	sql-eventplanner-prod
Azure Redis	redis-{app}-{env}	redis-eventplanner-prod
Service Bus Namespace	sb-{app}-{env}	sb-eventplanner-prod
Storage Account	st{app}{env}	steventplannerprod
AI Search	srch-{app}-{env}	srch-eventplanner-prod
Cosmos DB Account	cosmos-{app}-{env}	cosmos-eventplanner-prod
Key Vault	kv-{app}-{env}	kv-eventplanner-prod
App Insights	appi-{app}-{env}	appi-eventplanner-prod
Container Registry	cr{app}{env}	creventplannerprod
SignalR Service	sigr-{app}-{env}	sigr-eventplanner-prod

7.2 Core Bicep Template (main.bicep)

```
param environment string                // dev | staging | prod

param location string = resourceGroup().location

param sqlAdminPassword string          // from Key Vault reference


// Container App for .NET API

module api './modules/container-app.bicep' = {

    name: 'api-deployment'

    params: {

        name: 'ca-eventplanner-api-${environment}'

        image: 'creventplanner.azurecr.io/api:${deploymentTag}'

        env: environment

        minReplicas: environment == 'prod' ? 2 : 1

        maxReplicas: 20

        secrets: [{ name: 'sqlconn', keyVaultUrl: kv.outputs.sqlConnSecret }]

    }

}
```

```
}

// Azure SQL with read replica in prod

module sql './modules/sql.bicep' = {

    name: 'sql-deployment'

    params: {

        serverName: 'sql-eventplanner-${environment}'

        databaseName: 'EventPlannerDb'

        sku: environment == 'prod' ? 'Hyperscale' : 'GeneralPurpose'

        enableReadReplica: environment == 'prod'

    }

}

// Service Bus with dead-letter topic per domain event

module sb './modules/service-bus.bicep' = {
```

```
name: 'sb-deployment'

params: {

  namespaceName: 'sb-eventplanner-${environment}'

  sku: environment == 'prod' ? 'Premium' : 'Standard'

  topics: ['bookings', 'payments', 'notifications', 'disputes']

}

}
```

7.3 Service Bus Topics & Subscriptions

Topic	Publisher	Subscriptions (Consumers)	DLQ Retention
bookings	Bookings Module	Notifications, Availability, Analytics	7 days
payments	Payments Module	Bookings (confirm), Escrow, Notifications	14 days
notifications	All modules	Notification Hub Worker	3 days
disputes	Disputes Module	Admin, Notifications, Settlement	14 days
vendor-events	Identity Module	Catalogue Indexer, Notifications	7 days
outbox-relay	Outbox Worker	Domain-specific topic routers	3 days

8. CI/CD Pipeline (Azure DevOps)

8.1 Pipeline Strategy

Branch Strategy (GitHub Flow)

- **main**: Production branch — protected, requires 2 approvals + all checks passing
- **develop**: Integration branch — auto-deploys to dev environment on merge
- **feature/EP-{ticket}-{slug}**: Feature branches — PR to develop, auto-cancelled after 14 days
- **hotfix/EP-{ticket}-{slug}**: Hotfix branches — PR directly to main + backport to develop
- **release/v{major}.{minor}**: Release branches — used for staging validation before main merge

8.2 Pipeline Stages

Stage	Trigger	Steps	Gate
CI — Build & Test	Every PR + push to develop/main	Restore, Build, Unit Tests, Integration Tests, ArchUnit Tests, SAST (SonarCloud), Docker Build	All tests green, coverage ≥ 80%, no critical SAST issues
CD — Dev Deploy	Merge to develop	Push image to ACR, Deploy to dev Container App, Run smoke tests, Seed dev data	Smoke test pass
CD — Staging Deploy	Merge to release/*	Deploy to staging, Run E2E tests (Playwright), Performance test (k6), OWASP ZAP scan	E2E green, p95 latency < 800ms, no high OWASP findings
CD — Prod Deploy	Merge to main + manual approval	Blue-green deploy to Container Apps, Health check, Swap traffic 10% → 50% → 100%, Smoke test	Manual gate: 2 approvers, SRE sign-off, post-deploy health check
DB Migrations	Part of CD stages	EF Core migrations run before app swap via migration container job	Migration idempotency verified, rollback script prepared

8.3 Pipeline YAML (Backend CI)

```
# .azuredevops/backend-ci.yml

trigger:

    branches: { include: [develop, main, feature/*, hotfix/*] }

variables:

    dotnetVersion: "10.x"

    buildConfig: Release

stages:

- stage: Build

    jobs:

    - job: BuildAndTest

        pool: { vmImage: ubuntu-latest }

        steps:

        - task: UseDotNet@2
```

```
inputs: { version: $(dotnetVersion) }
```

```
- script: dotnet restore
```

```
displayName: Restore packages
```

```
- script: dotnet build --no-restore -c $(buildConfig)
```

```
displayName: Build
```

```
- script: dotnet test --no-build -c $(buildConfig)
```

```
--collect "XPlat Code Coverage" --logger trx
```

```
displayName: Unit + Integration Tests
```

```
- task: PublishTestResults@2
```

```
inputs: { testResultsFormat: VSTest, testResultsFiles: "**/*.trx" }
```

```
- task: PublishCodeCoverageResults@1
```

```
inputs: { codeCoverageTool: Cobertura, summaryFileLocation: "coverage.xml" }
```

```
- task: Docker@2
```

```
displayName: Build & Push Docker Image
```

```
inputs:
```

```
containerRegistry: $(acrServiceConnection)
```

```
repository: eventplanner/api
```

```
command: buildAndPush
```

```
tags: $(Build.BuildId)
```

9. Security Implementation Checklist

9.1 Authentication & Authorisation

Item	Implementation	Priority
JWT Access Tokens	Issued by Azure AD B2C, 15-minute lifetime, RS256 signed	P0
Refresh Tokens	Rotating refresh tokens, 7-day lifetime, single-use	P0
Token Revocation	Redis blocklist for revoked JTIs on logout/suspicious activity	P0
RBAC	Claims-based: Role = User Vendor Admin SuperAdmin	P0
Resource-level Auth	Users can only access their own events/bookings (owner check in handlers)	P0
OTP Verification	Phone verified before any booking. 6-digit, 5-minute TTL, max 3 attempts	P0
Social SSO	Google + Apple via Azure AD B2C external identity providers	P1
MFA for Admin	Admin portal requires MFA via Azure AD B2C conditional access	P0

9.2 API Security

Item	Implementation	Priority
Rate Limiting	Azure APIM: 100 req/min per IP, 1000/min per authenticated user	P0
Input Validation	FluentValidation on ALL commands; APIM schema validation on entry	P0
SQL Injection	EF Core parameterised queries only. Raw SQL prohibited in code review	P0
Webhook Signature	HMAC-SHA256 validation on all payment gateway webhooks	P0
CORS	Strict origin whitelist — only portal domains allowed	P0
HTTPS Only	Azure Front Door enforces HTTPS. HTTP → 301 redirect	P0
WAF Rules	Azure Front Door WAF: OWASP 3.2 rule set + custom rules for payment endpoints	P0
Secrets Management	All connection strings in Azure Key Vault. Never in appsettings or env vars in code	P0
Blob SAS Tokens	Short-lived SAS tokens (1 hour) for user file uploads. No public blob containers	P0
DDoS Protection	Azure DDoS Standard on the virtual network. Front Door absorbs volumetric attacks	P1

9.3 Data Protection

PII & Financial Data Handling

- **Encrypt at rest:** Azure SQL TDE (enabled by default), Cosmos DB encryption, Key Vault-managed keys
- **Encrypt in transit:** TLS 1.2 minimum across all service-to-service connections
- **PII masking in logs:** Serilog destructuring policies mask email, phone, and payment references in all log output
- **Payment card data:** Never stored locally. All card data handled by PCI-compliant payment gateway
- **GDPR compliance:** User data deletion endpoint within 30 days. Data export available. Consent recorded

10. Testing Strategy

Test Type	Tool	Target Coverage	Runs In
Unit Tests	xUnit + FluentAssertions + Bogus	Domain logic, handlers, validators ≥ 90%	Every PR (< 2 min)
Integration Tests	xUnit + Testcontainers-dotnet	API endpoints with real SQL/Redis ≥ 70%	Every PR (< 8 min)
Architecture Tests	ArchUnitNET	Module boundary enforcement, no circular deps	Every PR (< 1 min)
Contract Tests	Pact.NET	Frontend ↔ Backend API contracts	Every PR
E2E Tests	Playwright + TypeScript	Critical user journeys: Register → Book → Pay	Staging deploy (< 15 min)
Performance Tests	k6	500 concurrent users, p95 < 800ms, booking flow	Staging deploy
Security Tests	OWASP ZAP + SonarCloud SAST	API endpoints, dependency audit	Staging deploy
Accessibility	axe-core + Playwright	WCAG 2.1 AA on all Angular views	Staging deploy

10.1 Integration Test Pattern (Testcontainers)

```
public class BookingIntegrationTests : IAsyncLifetime

{

    private readonly MsSqlContainer _sqlContainer = new MsSqlBuilder()

        .WithImage("mcr.microsoft.com/mssql/server:2022-latest")

        .Build();

    private readonly RedisContainer _redisContainer = new RedisBuilder().Build();

    private WebApplicationFactory<Program> _factory;

    public async Task InitializeAsync()

    {

        await _sqlContainer.StartAsync();

        await _redisContainer.StartAsync();

        _factory = new WebApplicationFactory<Program>().WithWebHostBuilder(b =>
```

```
{

    b.ConfigureServices(s => {

        s.AddDbContext<AppDbContext>(o =>

            o.UseSqlServer(_sqlContainer.GetConnectionString()));

        s.AddStackExchangeRedisCache(o =>

            o.Configuration = _redisContainer.GetConnectionString());

        });

    });

}
```

[Fact]

```
public async Task CreateBooking_ValidRequest_ReturnsCreated()
```

```
{

    var client = _factory.CreateClient();

    client.DefaultRequestHeaders.Authorization =
```

```
new AuthenticationHeaderValue("Bearer", await GetTestJwt("user"));

var response = await client.PostAsJsonAsync("/api/v1/bookings", new {

    EventId = Guid.NewGuid(), VendorId = Guid.NewGuid(),

    ServiceId = Guid.NewGuid(), SubServiceIds = Array.Empty<Guid>(),

    EventDate = DateOnly.FromDateTime(DateTime.UtcNow.AddDays(30))

}, new HttpRequestMessage().Headers.Add("Idempotency-Key", Guid.NewGuid().ToString()));

response.StatusCode.Should().Be(HttpStatusCode.Created);

}

}
```

11. Development Sprint Plan

Based on a 2-week sprint cadence with a team of 3 backend developers, 2 frontend developers, 1 DevOps engineer, and 1 QA engineer. Total estimated timeline to production-ready MVP: 24 weeks (12 sprints).

11.1 Sprint Breakdown

Sprint	Weeks	Theme	Key Deliverables	Done When
S1–S2	1–4	Foundation	Azure infra (Bicep), CI/CD pipeline, monorepo setup, DB schema v1, auth module, APIM config, dev environment runbook	Engineers can run full stack locally and deploy to dev
S3–S4	5–8	Vendor Platform	KYC flow, background check integration, service/sub-service catalogue, availability calendar, portfolio upload, vendor portal Angular shell	A vendor can register, get approved, and publish services
S5–S6	9–12	User Booking Flow	Event creation, multi-vendor search + filter, sub-service selector, slot locking (Redis Redlock), order summary, contract generation, booking state machine	A user can search, select, and create bookings for an event
S7–S8	13–16	Payments & Escrow	Payment gateway integration, outbox pattern, escrow module, refund policy engine (configurable), payment ledger, idempotency, webhook handling	End-to-end payment + escrow for a booking works reliably
S9	17–18	Event Execution	Vendor check-in PWA, real-time SignalR dashboard, event day timeline, SLA monitoring, auto-escalation Durable Function, deliverable upload	Vendor can check in and users see live status
S10	19–20	Post-Event	Dispute module (all states), admin arbitration portal, escrow release, settlement + payout, commission config, tax calculation, invoice PDF	Full post-event flow works including dispute resolution
S11	21–22	Reviews & Admin	Multi-dimension review, moderation queue, vendor response, rating → search rank feed, admin dashboard (all	Reviews live, admin can manage full platform

			12 capabilities), fraud detection integration	
S12	23–24	Hardening & Launch	Security pen test, performance testing (k6), E2E Playwright suite, load test (500 concurrent), OWASP ZAP, accessibility audit, SLO dashboards, prod deploy, go-live runbook	Production deploy with SLO dashboards live

12. Non-Functional Requirements (SLOs)

Requirement	Target	Measurement	Alert Threshold
API Availability	99.9% monthly uptime	Azure Monitor synthetic tests every 1 min	< 99.5% in any 24h window
API Response Time (p95)	< 400ms for reads, < 800ms for writes	App Insights percentile metrics	p95 > 1000ms for > 5 min
Payment Webhook Processing	< 30 seconds end-to-end	Custom metric from webhook receipt to escrow update	> 60 seconds on any webhook
Search Response	< 200ms (p95)	Azure AI Search latency + Redis cache hit rate	p95 > 500ms or cache hit < 80%
Slot Lock Acquisition	< 100ms (p95)	Redis operation latency metric	> 300ms or lock failure rate > 1%
Notification Delivery	< 60 seconds after trigger event	Notification Hub delivery reports	> 5% failure rate on any channel
DB Write Throughput	500 concurrent writes/second	SQL DTU / vCore metrics	> 80% DTU sustained for > 10 min
Error Rate	< 0.1% of all API calls (5xx)	App Insights failure rate	> 1% in any 5-minute window
Data Recovery (RTO)	< 4 hours	Defined in DR runbook, tested quarterly	DR drill failure
Data Backup (RPO)	< 1 hour data loss	Azure SQL automated backups every 12h + log shipping	Backup failure alert

12.1 Observability Setup

Azure Monitor Stack

- **App Insights SDK:** Instrument every .NET 10 API and Angular app for distributed tracing, custom events, and dependency tracking
- **Log Analytics Workspace:** Centralised log aggregation from all services with KQL query dashboards for operations team
- **SLO Dashboard (Power BI / Grafana):** Real-time dashboard showing: booking conversion rate, payment success rate, vendor SLA compliance, average dispute resolution time
- **PagerDuty Integration:** Azure Monitor alert rules → PagerDuty → on-call rotation for P0 incidents (payment failures, DB downtime, error rate spikes)
- **Synthetic Tests:** Azure Monitor availability tests hitting /health endpoint and a critical booking path every minute from 3 Azure regions

13. Developer Environment Setup

13.1 Prerequisites

Tool	Version	Purpose	Install
.NET SDK	10.x	Backend development	dotnet.microsoft.com
Node.js	22 LTS	Angular frontend	nodejs.org
Angular CLI	20.x	Angular development	npm install -g @angular/cli@20
Docker Desktop	Latest	Testcontainers, local SQL/Redis	docker.com
Azure CLI	2.x	Azure resource management	docs.microsoft.com/cli
Azure Bicep	0.x	IaC deployment	az bicep install
VS Code	Latest	Recommended IDE	code.visualstudio.com
Azure Data Studio	Latest	SQL management	Optional — for DB work

13.2 Local Setup Steps

1. Clone the monorepo: git clone https://dev.azure.com/org/event-planner.git

2. Copy .env.template to .env and fill in Azure AD B2C tenant details (from team password manager)
3. Start local dependencies: docker compose up -d (starts SQL Server 2022 + Redis + Azurite)
4. Run EF Core migrations: cd src/backend && dotnet ef database update --project EventPlanner.Infrastructure
5. Seed development data: dotnet run --project scripts/Seed -- --env dev
6. Start the backend API: dotnet run --project src/EventPlanner.Api (runs on http://localhost:5001)
7. Start the user portal: cd src/frontend && ng serve user-portal (runs on http://localhost:4200)
8. Start the vendor portal: ng serve vendor-portal (runs on http://localhost:4201)
9. Start the admin portal: ng serve admin-portal (runs on http://localhost:4202)

13.3 docker-compose.yml (Local Dependencies)

```
services:
```

```
  sql:
```

```
    image: mcr.microsoft.com/mssql/server:2022-latest
```

```
    environment:
```

```
      SA_PASSWORD: "EventPlanner@Local123"
```

```
      ACCEPT_EULA: "Y"
```

```
    ports: ["1433:1433"]
```

```
    volumes: [sql-data:/var/opt/mssql]
```

```
  redis:
```

```
    image: redis:7-alpine
```

```
    ports: ["6379:6379"]
```

```
    command: redis-server --requirepass "localredispass"
```

```
  azurite: # Azure Storage emulator
```

```
image: mcr.microsoft.com/azure-storage/azurite
```

```
ports: ["10000:10000", "10001:10001", "10002:10002"]
```

```
elasticsearch:      # Local search (replaces Azure AI Search in dev)
```

```
image: elasticsearch:8.12.0
```

```
environment: { discovery.type: single-node, xpack.security.enabled: "false" }
```

```
ports: ["9200:9200"]
```

```
volumes:
```

```
sql-data:
```

14. Architecture Decision Records (ADRs)

The following decisions have been made and documented. All future architectural decisions must follow the ADR format and be stored in `/docs/adr/`.

ADR #	Decision	Rationale	Trade-offs Accepted
ADR-001	Modular Monolith over Microservices	Team size < 10. Operational complexity of microservices outweighs benefits at launch. Module boundaries enforced by ArchUnit.	Single deployment unit. Extract to services when team/load demands.
ADR-002	Vertical Slice Architecture	Keeps all code for a feature co-located. Eliminates cross-layer navigation. Easier onboarding.	Some code duplication across slices (acceptable — prefer duplication over wrong abstraction).
ADR-003	Outbox Pattern for all async messages	Guarantees at-least-once delivery of domain events. Prevents lost events on app crash mid-transaction.	Extra outbox table and background worker. Slight processing latency (~5s).
ADR-004	Redis Redlock for slot locking	Prevents double-booking under concurrent load. 15-minute TTL auto-releases abandoned locks.	Redis is now a critical dependency for bookings. Must be highly available (cluster mode).
ADR-005	Cosmos DB for audit log only	Immutable, append-only audit trail benefits from Cosmos schema flexibility and Change Feed.	Adds Cosmos operational complexity. Justified by compliance and dispute evidence needs.
ADR-006	Cursor-based pagination	Offset pagination degrades at scale (OFFSET 10000 is slow). Cursor is O(log n).	Cannot jump to arbitrary pages. UI must use infinite scroll or next/prev only.
ADR-007	Angular Signals over NgRx	Angular 20 Signals API provides built-in reactivity without NgRx boilerplate for most use cases.	Use NgRx Signal Store for complex cross-component state only (booking cart).
ADR-008	Azure AD B2C for all auth	Managed auth removes custom token logic. Supports OTP, SSO, MFA out of the box.	Vendor lock-in to Azure AD B2C. Migration cost if switching providers.

15. Definition of Done

Every user story must meet ALL of the following criteria before it can be marked Done:

- Code reviewed by at least 1 peer (2 for payment/auth changes)
- Unit tests written and passing ($\geq 90\%$ coverage on new code)
- Integration tests added for new API endpoints
- Architecture boundary tests still passing (ArchUnit)
- No new SonarCloud critical or blocker issues
- API endpoint documented in OpenAPI spec
- Database migration is idempotent and has rollback script
- Feature deployed to dev environment and smoke-tested
- Acceptance criteria from ticket verified by QA
- New Azure resources added to Bicep templates
- No secrets committed to repository (pre-commit hook enforced)
- PII fields masked in all log statements
- Responsive UI tested on mobile viewport (vendor PWA)
- Ticket updated with deployment notes and test evidence